

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گروه مهندسی نرم افزار

پایان نامه کارشناسی

رشته مهندسی کامپیوتر گرایش نرم افزار

عنوان پایان نامه

بهره‌گیری از مدل‌های زبانی بزرگ برای تحلیل خودکار نتایج تست

استاد راهنما:

دکتر محمدرضا شعراف

پژوهشگر:

محدثه آخوندی خضرآباد

شهریور ۱۴۰۴



دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گروه مهندسی نرم افزار

پروژه کارشناسی رشته‌ی مهندسی کامپیوتر گرایش نرم افزار

خانم محدثه آخوندی خضرآباد

تحت عنوان

بهره‌گیری از مدل‌های زبانی بزرگ برای تحلیل خودکار نتایج تست

در تاریخ ۱۴۰۴ / ۶ / توسط هیأت داوران زیر بررسی و با نمره به تصویب نهایی رسید.

۱- استاد راهنمای پروژه:

دکتر محمدرضا شهرباف

امضا

۲- استاد داور :

دکتر

امضا

امضای مدیر گروه

تشکر و قدردانی

از استاد گرامی خودم جناب آقای دکتر محمدرضا شهرباف که با راهنمایی‌های ارزشمند خود مسیر انجام این پروژه را هموار ساختند صمیمانه قدردانی می‌کنم. همچنین از دوستان عزیزم که در طول این مسیر با حمایت‌ها و همراهی‌هایشان یاری‌گر من بودند، سپاسگزارم.

تقدیم به پدر و مادر عزیزم؛

که آرامش نگاهشان، پناهگاه من و دعای خیرشان، چراغ راه من بوده
است؛ این تلاش کوچک را به شما تقدیم می‌کنم، به پاس سال‌ها مهر،
صبوری و ایستادگی‌تان در کنارم.

چکیده:

در چرخه توسعه نرم افزار، آزمون های خودکار نقشی کلیدی در شناسایی سریع خطاها و تضمین کیفیت کد ایفا می کنند. با این حال، افزایش ابعاد سامانه ها و پیچیدگی وابستگی های درونی، منجر به تولید حجم عظیمی از گزارش ها و لاگ های تست می شود که تحلیل دستی آن ها زمان بر و مستعد خطاست. یکی از مهم ترین چالش ها، شناسایی سریع و دقیق مسیرهای بحرانی و علل شکست تست ها در میان انبوه داده ها است. در این پروژه، سامانه ای هوشمند برای تحلیل خودکار نتایج آزمون های نرم افزاری با استفاده از مدل های زبانی بزرگ طراحی و پیاده سازی شده است. این سامانه قادر است خروجی ابزارهای رایج تست را دریافت و پس از نرمال سازی، وابستگی های کد را در قالب یک گراف جهت دار مدل سازی کند. مسیرهای بحرانی مرتبط با تست های شکست خورده استخراج شده و همراه با متاداده ها، خلاصه تست ها و نمونه های آموزشی در قالب پرامپت های ساختارمند به مدل زبانی ارسال می شوند. مدل زبانی با تحلیل این داده ها، علل محتمل شکست تست، الگوهای تکرارشونده و پیشنهادهای اصلاحی را در قالبی قابل استفاده برای تیم توسعه ارائه می دهد. معماری ماژولار سامانه، پشتیبانی از زبان های برنامه نویسی مختلف از جمله Python، Java و C# را ممکن ساخته و امکان یکپارچه سازی آن با خطوط یکپارچه استقرار و تحویل مداوم را برای تحلیل بلادرنگ نتایج فراهم کرده است. نتایج ارزیابی ها نشان می دهد که این رویکرد زمان شناسایی و تحلیل خطا را به طور قابل توجهی کاهش داده و دقت تشخیص علل شکست تست ها را افزایش می دهد. این سامانه می تواند به عنوان ابزاری عملیاتی در فرآیند تضمین کیفیت و نگهداری نرم افزار به کار گرفته شود.

واژگان کلیدی: آزمون نرم افزار، تحلیل تست نرم افزار، تحلیل ریشه ای خطا، مدل های زبانی بزرگ

فهرست مطالب

عنوان	صفحه
فصل اول مقدمه	۱
۱-۱- بیان مسئله	۱
۲-۱- هدف پروژه	۲
۳-۱- کاربردهای پروژه	۲
۴-۱- ساختار پایان نامه	۳
فصل دوم مفاهیم	۴
۱-۲- مقدمه	۴
۲-۲- معماری کلی سامانه	۴
۱-۲-۲- ساختار کلی اجزای سامانه	۵
۲-۲-۲- جریان داده و تعامل بخش ها	۵
۳-۲-۲- مدیریت نسخه و انتشار	۵
۳-۲- فناوری و ابزارهای توسعه	۶
۱-۳-۲- پایتون	۶
۲-۳-۲- رابط کاربری و ارائه نتایج	۷
۱-۲-۳-۲- HTML	۷
۲-۲-۳-۲- CSS	۷
۳-۲-۳-۲- جاوااسکریپت	۸
۴-۲-۳-۲- Tailwind CSS	۸
۵-۲-۳-۲- کتابخانه های مصورسازی داده	۹
۳-۳-۲- ابزارهای خط فرمان	۱۰
۴-۲- مدل های زبانی بزرگ	۱۱
۱-۴-۲- معرفی و مبانی نظری	۱۱
۲-۴-۲- روشهای تعامل و به کارگیری	۱۱
۳-۴-۲- کاربردها در تحلیل و پردازش داده	۱۱
۵-۲- گراف و پردازش داده های آزمون	۱۲
۱-۵-۲- مفاهیم گراف وابستگی و ساختار آن در سامانه	۱۲

فهرست مطالب

صفحه	عنوان
۱۲-۵-۲	نقش گراف در نمایش روابط بین اجزای نرم افزار.....
۱۲-۵-۳	مفاهیم استانداردسازی داده های آزمون و نقش آن در تحلیل.....
۱۲-۵-۴	نگاشت مفهومی داده های آزمون بر روی گراف و اهمیت آن در تصمیم گیری.....
۱۳-۶-۲	چارچوب و سرویس های پشتیبان.....
۱۳-۶-۱	مدیریت نسخه و انتشار پروژه.....
۱۳-۶-۲	انتشار بسته نرم افزاری در PyPI با Poetry.....
۱۴-۶-۳	سرویس های خارجی.....
۱۴-۶-۱-۳	سرویس AvalAI.....
۱۵-۷-۲	جمع بندی.....
۱۶-۸-۱	فصل سوم پیشینه پژوهشی.....
۱۶-۳-۱	مقدمه.....
۱۶-۳-۲	معرفی کارهای مرتبط.....
۱۶-۳-۱-۲	مطالعه و بررسی مقاله مرتبط.....
۱۷-۳-۲-۲	روش های تحلیل و پایش نتایج تست های نرم افزاری.....
۱۸-۳-۳	معرفی و ارزیابی ابزارهای منتخب.....
۱۸-۳-۱-۳	TestComplete.....
۱۹-۳-۲-۳	TestRail.....
۲۰-۳-۳-۳	ReportPortal.....
۲۱-۳-۴-۳	ReportNG.....
۲۲-۳-۵-۳	Allure Report.....
۲۳-۳-۶-۳	Allure TestOps.....
۲۴-۳-۴	تحلیل و مقایسه ابزارها.....
۲۵-۳-۵	ضرورت ایجاد ابزاری که در این پژوهش پیاده سازی شد.....
۲۶-۳-۶	جمع بندی.....
۲۷-۴-۱	فصل چهارم راهکار پیشنهادی.....
۲۷-۴-۱	مقدمه.....
۲۷-۴-۲	فهرست نیازمندی ها.....

فهرست مطالب

صفحه	عنوان
۲۷	۴-۲-۱- نیازمندی‌های کارکردی
۲۸	۴-۲-۲- نیازمندی‌های غیرکارکردی
۲۹	۴-۳- معماری سیستم
۳۰	۴-۴- روند کلی سامانه
۳۱	۴-۵- پیاده‌سازی سیستم
۳۱	۴-۵-۱- پیاده‌سازی ماژول CLI
۳۵	۴-۵-۲- پیاده‌سازی ماژول تحلیل و گراف کد
۳۸	۴-۵-۳- پردازش لاگ‌های تست و نرمال‌سازی
۳۹	۴-۵-۴- استخراج مسیرهای بحرانی و توابع کلیدی
۴۱	۴-۵-۵- ساخت پرامیت ساختاریافته و ارتباط با LLM
۴۶	۴-۵-۶- تولید گزارش نهایی
۴۹	۴-۵-۷- نحوه نصب و اجرای نرم‌افزار
۵۰	۴-۵-۷-۱- آماده‌سازی محیط اجرا
۵۰	۴-۵-۷-۲- نصب ابزار Failtrace
۵۱	۴-۵-۷-۳- اجرای نرم‌افزار
۵۲	۴-۵-۷-۴- توضیح پارامترها
۵۲	۴-۶- جمع‌بندی
۵۳	فصل پنجم طرح تجاری
۵۳	۵-۱- مقدمه
۵۳	۵-۲- جامعه هدف
۵۴	۵-۳- سودرسانی
۵۵	۵-۴- استراتژی بازاریابی و تبلیغات
۵۵	۵-۴-۱- معرفی و ترویج از طریق محتوای آموزشی و شبکه‌های تخصصی
۵۵	۵-۴-۲- همکاری با شتاب‌دهنده‌ها و شرکت‌های نرم‌افزاری
۵۶	۵-۵- جمع‌بندی
۵۷	فصل ششم نتیجه‌گیری و پیشنهادها

فهرست مطالب

صفحه	عنوان
۵۷.....	۶-۱- خلاصه
۵۸.....	۶-۲- چالش‌ها و محدودیت‌ها
۵۸.....	۶-۳- کارهای آینده
۶۰.....	منابع
۶۱.....	پیوست ۱: لیست برنامه‌ها

فهرست شکل‌ها

عنوان	صفحه
شکل ۴-۱: معماری سامانه تحلیل نتایج آزمون.....	۲۹
شکل ۴-۲: فرآیند کلی پردازش و تحلیل داده‌های آزمون در سامانه.....	۳۰
شکل ۴-۳: نمودار توالی اجرای فرآیند تحلیل خطا در سامانه Failtrace.....	۳۱
شکل ۴-۴: اجرای ابزار Failtrace در حالت کامل از طریق خط فرمان.....	۳۳
شکل ۴-۵: اجرای ابزار Failtrace در حالت سریع از طریق خط فرمان.....	۳۴
شکل ۴-۶: نمونه‌ای از اجرای ابزار Failtrace در حالت کامل از طریق خط فرمان.....	۳۵
شکل ۴-۷: نمونه‌ای از گراف وابستگی تولیدشده برای یک پروژه نرم‌افزاری.....	۳۶
شکل ۴-۸: کد detector.py.....	۳۷
شکل ۴-۹: کد plugin.py.....	۳۷
شکل ۴-۱۰: ساختار فایل summary.json شامل داده‌های لاگ آزمون.....	۳۸
شکل ۴-۱۱: خروجی استخراج توابع کلیدی در مسیرهای بحرانی یک پروژه نرم‌افزاری.....	۴۰
شکل ۴-۱۲: نمونه خروجی استخراج نهایی پرامپت تولید شده توسط سامانه بخش اول.....	۴۱
شکل ۴-۱۳: نمونه خروجی استخراج نهایی پرامپت تولید شده توسط سامانه بخش دوم.....	۴۲
شکل ۴-۱۴: نمونه خروجی نهایی ساخت پرامپت برای ارسال به مدل زبانی بخش اول.....	۴۳
شکل ۴-۱۵: نمونه خروجی نهایی ساخت پرامپت جهت ارسال به مدل زبانی بخش دوم.....	۴۳
شکل ۴-۱۶: نمونه خروجی نهایی ساخت پرامپت جهت ارسال به مدل زبانی بخش سوم.....	۴۴
شکل ۴-۱۷: نمونه خروجی نهایی ساخت پرامپت جهت ارسال به مدل زبانی بخش چهارم.....	۴۴
شکل ۴-۱۸: کد ماژول avalai_client.py.....	۴۵
شکل ۴-۱۹: نمودار آماری Doughnut Chart برای تحلیل نتایج یک پروژه نرم‌افزاری.....	۴۶
شکل ۴-۲۰: نمودار آماری میله‌ای (Bar Chart) برای نتایج آزمون در یک پروژه نرم‌افزاری.....	۴۷
شکل ۴-۲۱: جدول Failed items در آزمون یک پروژه نرم‌افزاری.....	۴۷
شکل ۴-۲۲: جدول اطلاعات تحلیل خطاها.....	۴۸
شکل ۴-۲۳: نمودار حبابی مصورسازی ماتریس ریسک.....	۴۹
شکل ۴-۲۴: نمایش وضعیت آماری کلی آزمون در یک پروژه نرم‌افزاری.....	۴۹
شکل ۴-۲۵: خروجی اجرای سامانه Failtrace در محیط خط فرمان.....	۵۱

فهرست جدول‌ها

صفحه	عنوان
۲۵	جدول ۱-۲: مقایسه ابزارها بر اساس معیارها.....
۵۲	جدول ۲-۴: پارامترها و گزینه‌های خط فرمان (CLI) سامانه Failtrace.....

مخفف‌ها:

LLM	Large Language Models
QA	Quality Assurance
CI/CD	Continuous Integration / Continuous Delivery
HTML	HyperText Markup Language
CLI	Command Line Interface
PyPI	Python Package Index
CSS	Cascading Style Sheets
API	Application Programming Interface
MDSE	Model-Driven Software Engineering

فصل اول

مقدمه

۱-۱- بیان مسئله

در پروژه‌های نرم‌افزاری مبتنی بر کد^۱ به‌ویژه در مقیاس‌های متوسط و بزرگ، اجرای مداوم تست‌های واحد و تست‌های یکپارچه سازی نقش حیاتی در تضمین کیفیت و پایداری سیستم دارد. این دسته از تست‌ها با هدف اعتبار سنجی منطق داخلی کد و اطمینان از صحت تعامل میان ماژول‌ها طراحی می‌شوند. با این حال، چالش اصلی در این حوزه نه صرفاً اجرای تست‌ها، بلکه تحلیل و پایش مؤثر نتایج آن‌ها است؛ چرا که حجم زیاد گزارش‌ها و تنوع خطاها، فرآیند شناسایی علل شکست را پیچیده و زمان‌بر می‌سازد. از این رو، استفاده از رویکردهای خودکار و هوشمند برای تحلیل داده‌های تست و استخراج مسیرهای بحرانی در کد به ضرورتی انکارناپذیر تبدیل شده است.

این پژوهش با تمرکز بر تحلیل و گزارش‌دهی تست‌های واحد و یکپارچه‌سازی، رویکردی سیستماتیک برای افزایش کارایی فرآیند تضمین کیفیت ارائه می‌دهد. سامانه پیشنهادی با بهره‌گیری از ابزارهای نوین گزارش‌گیری و ترکیب آن با مدل‌های زبانی پیشرفته، امکان نرمال‌سازی داده‌های تست، تحلیل خودکار مسیرهای اجرایی و شناسایی ریشه‌ای خطاها را فراهم می‌سازد. نتیجه‌ی نهایی این پروژه نه تنها به تیم‌های توسعه کمک می‌کند تا سریع‌تر به علل شکست تست‌ها دست یابند، بلکه به شرکت‌های نرم‌افزاری و سازمان‌هایی که به بهبود کیفیت محصولات خود وابسته‌اند، ابزاری عملی و کاربردی ارائه می‌دهد. چنین رویکردی می‌تواند به‌طور مستقیم در کاهش هزینه‌های توسعه، افزایش اعتمادپذیری نرم‌افزار و تسریع فرآیند تصمیم‌گیری در چرخه عمر محصول مؤثر واقع شود.

^۱ Code-based

۱-۲- هدف پروژه

هدف اصلی این پژوهش، ارائه رویکردی نوین برای تحلیل و گزارش‌دهی نتایج تست‌های واحد و یکپارچه‌سازی در پروژه‌های نرم‌افزاری مبتنی بر کد است. تمرکز اصلی بر آن است که با ترکیب ابزارهای گزارش‌گیری پیشرفته و^۱ LLM، چارچوبی هوشمند و خودکار برای شناسایی ریشه‌ای خطاها و پایش کارآمد فرآیند تست ایجاد شود. چنین رویکردی نه تنها دقت تحلیل را افزایش می‌دهد، بلکه با کاهش بار کاری تیم‌های توسعه، امکان واکنش سریع‌تر به شکست‌ها و بهبود مداوم کیفیت نرم‌افزار را فراهم می‌سازد.

از سوی دیگر، این پژوهش به دنبال آن است که با معرفی سازوکاری سیستماتیک برای نرمال‌سازی داده‌های تست، استخراج مسیرهای بحرانی و ارائه گزارش‌های ساخت‌یافته، بستری فراهم کند که بتواند در مقیاس سازمانی نیز مورد استفاده قرار گیرد. دستاورد نهایی پروژه، ابزاری کاربردی خواهد بود که به تیم‌های توسعه و شرکت‌های نرم‌افزاری کمک می‌کند ضمن کاهش هزینه‌های ناشی از تحلیل‌های دستی و زمان‌بر، سطح اطمینان و اعتمادپذیری محصولات خود را ارتقا دهند. این دستاورد به‌طور مستقیم می‌تواند در خدمت صنایع و کسب‌وکارهایی قرار گیرد که کیفیت نرم‌افزار و سرعت عرضه به بازار برای آن‌ها حیاتی است.

۱-۳- کاربردهای پروژه

سامانه طراحی‌شده در این پروژه می‌تواند در حوزه‌های مختلف توسعه و نگهداری نرم‌افزار به‌کار گرفته شود و به‌ویژه برای توسعه‌دهندگان و تسترها ارزش عملیاتی بالایی داشته باشد. این سامانه با امکان تحلیل خودکار و دقیق نتایج آزمون‌ها، فرآیند شناسایی و رفع خطا را تسریع کرده و موجب کاهش بار کاری تیم‌های تضمین کیفیت^۲ و تحلیل‌گران تست می‌شود. از سوی دیگر، برای توسعه‌دهندگان نیز این قابلیت فراهم می‌گردد که سریع‌تر به نقاط شکست کد دست یابند و بازخورد فوری نسبت به کیفیت تغییرات خود دریافت کنند.

در محیط‌های توسعه نرم‌افزار که از خطوط یکپارچه استقرار و تحویل مداوم^۳ بهره می‌برند، این ابزار قادر است گزارش‌ها را به‌صورت بلادرنگ پردازش کرده و تحلیل علل شکست تست‌ها را در همان چرخه‌ی استقرار ارائه دهد. همچنین، با پشتیبانی از زبان‌های برنامه‌نویسی مختلف نظیر پایتون^۴، جاوا^۵ و سی‌شارپ^۶ امکان استفاده در پروژه‌های چندزبانه فراهم می‌شود و وابستگی‌ها و مسیرهای بحرانی حتی در سامانه‌های ناهمگون قابل شناسایی خواهد بود. این ویژگی، سامانه را به ابزاری کارآمد برای توسعه‌دهندگان و تسترها در سازمان‌هایی

^۱ Large Language models

^۲ Quality Assurance

^۳ Continuous Integration /Continuous Delivery

^۴ Python

^۵ Java

^۶ C#

با پروژه‌های بزرگ و توزیع شده تبدیل می‌کند و نقش مهمی در ارتقاء کیفیت نرم‌افزار، کاهش زمان رفع نقص و افزایش بهره‌وری تیم‌های فنی ایفا می‌نماید.

۱-۴- ساختار پایان نامه

روند تنظیم این پایان‌نامه به صورت مرحله‌ای و نظام‌مند تدوین شده است. در فصل نخست، کلیات پژوهش شامل بیان مسئله، اهداف و کاربردهای پروژه مطرح گردیده است. فصل دوم به معرفی مفاهیم و مبانی نظری اختصاص دارد و در آن موضوعاتی همچون آزمون نرم‌افزار، مدل‌های زبانی بزرگ، گراف وابستگی کد و ابزارهای پشتیبان بررسی شده‌اند. در فصل سوم، پیشینه‌ی پژوهش مرور شده و کارهای مرتبط و جایگاه تحقیق حاضر نسبت به مطالعات پیشین تبیین گردیده است. فصل چهارم به شرح پروژه اختصاص یافته و در آن معماری سامانه، اجزای اصلی و روند پیاده‌سازی همراه با نمونه نتایج ارائه شده است. در فصل پنجم، ابعاد تجاری‌سازی سامانه شامل جامعه هدف، مدل سودرسانی و استراتژی‌های بازاریابی بیان گردیده است. نهایتاً در فصل ششم، جمع‌بندی کلی پژوهش همراه با چالش‌ها، محدودیت‌ها و پیشنهادهایی برای ادامه کار ارائه می‌شود.

فصل دوم

مفاهیم

۲-۱- مقدمه

یکی از گام‌های اساسی در اجرای هر پژوهش، تبیین مفاهیم و ابزارهایی است که بنیان کار بر آن‌ها استوار است. در این فصل، مجموعه‌ای از مباحث و فناوری‌های مورد نیاز برای طراحی و پیاده‌سازی سامانه معرفی و توضیح داده می‌شوند تا زمینه‌ای روشن برای درک بهتر پروژه فراهم گردد. ابتدا ساختار کلی سامانه و نحوه مدیریت فرایند تحلیل بیان می‌شود. سپس روش‌های تحلیل کد و تولید گراف وابستگی معرفی شده و اهمیت آن‌ها در شناسایی روابط میان اجزای نرم‌افزار تشریح می‌گردد. در ادامه، ابزارهای مرتبط با جمع‌آوری و استانداردسازی داده‌های آزمون توضیح داده شده و نقش آن‌ها در آماده‌سازی داده‌ها برای تحلیل بررسی می‌شود. همچنین به توانایی‌های مدل‌های زبانی در پردازش داده‌های متنی و کاربرد آن‌ها در تحلیل دقیق‌تر نتایج پرداخته خواهد شد. این فصل در مجموع به‌عنوان بستری نظری برای درک بهتر مراحل طراحی و پیاده‌سازی سامانه در فصل‌های بعد عمل می‌کند.

۲-۲- معماری کلی سامانه

سامانه به‌صورت ماژولار^۱ طراحی شده است تا هر جزء، مسئولیتی روشن و قابل آزمون داشته باشد. نقطه‌ی ورود، اپلیکیشن خط فرمان است که سناریوهای اجرا را مدیریت می‌کند، مسیر پروژه و گزارش‌های آزمون را دریافت می‌کند و اجرای مراحل را هماهنگ می‌سازد. لایه‌ی تحلیل کد، منابع پروژه را پیمایش می‌کند و با استفاده از ابزارهای تحلیل ایستا، موجودیت‌های قابل اجرا (مانند توابع، متدها و تست‌ها) و ارتباط‌های آن‌ها را استخراج می‌کند. خروجی این لایه یک گراف جهت‌دار است که در آن هر گره نماینده‌ی یک واحد رفتاری و هر یال نماینده‌ی یک وابستگی یا فراخوانی است. به‌صورت موازی، لایه‌ی پردازش آزمون، گزارش‌های تولیدشده

^۱ Modular Design

توسط چارچوب‌های رایج را به قالبی یکنواخت تبدیل می‌کند تا وضعیت اجرای هر تست و پیام‌های خطا قابل پردازش ماشینی باشد. لایه‌ی نگاشت، داده‌های آزمون استاندارد شده را به گره‌های متناظر در گراف نسبت می‌دهد تا بتوان مسیرهای بحرانی مرتبط با شکست‌ها را شناسایی کرد. در گام بعد، لایه‌ی مهندسی پرامپت با اتکا به خلاصه‌ی تست‌ها، مسیرهای بحرانی و برش‌های کد مرتبط، بسته‌ی داده‌ی ساختاریافته‌ای می‌سازد و آن را برای مدل زبانی ارسال می‌کند. در انتها، لایه ارائه، نتایج پردازش را در قالب یک گزارش تعاملی مبتنی بر وب در اختیار کاربر قرار می‌دهد.

۲-۲-۱- ساختار کلی اجزای سامانه

ساختار کلان شامل چند زیرسامانه است: استخراج ساختار کد و گراف، استانداردسازی و خلاصه‌سازی نتایج آزمون، نگاشت آزمون به گراف، تحلیل مدل‌های زبانی بزرگ و ارائه گزارش. هر زیرسامانه از طریق قراردادهای داده‌ای مشخص با دیگر اجزا تبادل می‌کند. این قراردادها شامل شناسه یکتا برای گره‌ها، قالب یکنواخت برای نتایج آزمون و طرح داده‌ی پرامپت ساختاریافته است. جداسازی وظایف باعث می‌شود هر جزء قابل جایگزینی و ارتقاء باشد؛ برای نمونه می‌توان چارچوب‌های آزمون جدید را بدون تغییر در منطق گراف اضافه کرد یا مدل زبانی را با حفظ قالب پرامپت به‌روزرسانی نمود.

۲-۲-۲- جریان داده و تعامل بخش‌ها

جریان داده از ورودی‌ها به خروجی‌های قابل مصرف کاربر به‌صورت مرحله‌ای پیش می‌رود. ابتدا منابع کد خوانده می‌شود و گراف وابستگی تولید می‌گردد. همزمان، گزارش‌های آزمون خوانده و به رکوردهای استاندارد شامل نام تست، وضعیت و پیام خطا تبدیل می‌شوند. سپس نگاشت مفهومی، نام‌های تست در گزارش‌ها را به شناسه‌های گره در گراف همسان می‌کند تا وضعیت اجرا روی گراف قابل نمایش و تحلیل باشد. مسیرهای پرتکرار یا مسیرهایی که با شکست‌ها هم‌بستگی دارند استخراج می‌شوند و خلاصه‌ی ساختاریافته‌ای از وضعیت پروژه شکل می‌گیرد. این خلاصه، همراه با نمونه‌های راهنما، در قالب پرامپت به مدل زبانی ارسال می‌شود. پاسخ مدل شامل تحلیل ریشه خطا و پیشنهاد‌های عملیاتی است که در کنار نمودارها و جداول در گزارش تعاملی نمایش داده می‌شود.

۲-۲-۳- مدیریت نسخه و انتشار

کد منبع در سامانه‌ی کنترل نسخه نگهداری می‌شود تا تاریخچه‌ی تغییرات و تکرارپذیری تحلیل‌ها تضمین گردد. فرایند انتشار به‌صورت بسته‌ی پایتون انجام می‌شود تا نصب و به‌روزرسانی ابزار برای کاربران نهایی ساده باشد. نسخه‌بندی معنایی و نگهداری وابستگی‌ها، امکان عرضه‌ی به‌روزرسانی‌های تدریجی را فراهم می‌کند و در

عین حال با حفظ قالب‌های داده، سازگاری رو به عقب را تضمین می‌نماید. این رویکرد، استقرار ابزار در محیط‌های مختلف و ادغام با خطوط CI/CD را تسهیل می‌کند.

۳-۲- فناوری و ابزارهای توسعه

۳-۲-۱- پایتون

پایتون یک زبان برنامه‌نویسی سطح بالا^۱، تفسیری و چندسکویی است که نخستین بار در سال ۱۹۹۱ توسط خیدو فان روسوم^۲ معرفی شد. این زبان با فلسفه سادگی در نوشتار، خوانایی بالا و بهره‌گیری از ساختارهایی شفاف طراحی شد و به مرور به یکی از پرکاربردترین زبان‌های برنامه‌نویسی در حوزه‌های گوناگون تبدیل گردید. پایتون از همان ابتدا با هدف تسهیل توسعه نرم‌افزار و کاهش پیچیدگی کدنویسی طراحی شده بود و همین ویژگی باعث شد تا به سرعت در جامعه دانشگاهی، صنعتی و تحقیقاتی جایگاه ویژه‌ای پیدا کند.

یکی از نقاط قوت اساسی پایتون، گستردگی کتابخانه‌ها و چارچوب‌های آن است. این زبان نه تنها برای توسعه نرم‌افزارهای کاربردی و سیستم‌های تحت وب به کار گرفته می‌شود، بلکه در حوزه‌های تحلیل داده، هوش مصنوعی، یادگیری ماشین و پردازش زبان طبیعی نیز جایگاه ممتازی دارد. وجود کتابخانه‌هایی همچون NumPy، Pandas، TensorFlow، scikit-learn باعث شده است تا پایتون به زبان مرجع پژوهشگران و مهندسان داده تبدیل شود [۱].

در سامانه حاضر، پایتون به عنوان زبان اصلی توسعه انتخاب شد. این انتخاب به دلیل چند ویژگی کلیدی پایتون بوده است: نخست، سادگی و خوانایی بالای کد که امکان همکاری تیمی و توسعه ماژولار را فراهم می‌آورد. دوم، پشتیبانی گسترده از ابزارهای تحلیل استاتیک و پویا مانند libst و tree-sitter که در پروژه برای استخراج و تحلیل گراف وابستگی کدها مورد استفاده قرار گرفتند. سوم، سازگاری با ابزارهای خط فرمان که نقش مهمی در توسعه اپلیکیشن CLI این سامانه داشت. در واقع، امکانات ذاتی پایتون در ساخت رابط‌های خط فرمانی از طریق ماژول‌هایی نظیر argparse یا کتابخانه‌های مشابه، موجب شد تا سامانه بتواند به صورت سبک، مستقل از بستر وب و در محیط کاربر اجرا گردد.

ویژگی‌های چندسکویی بودن پایتون، جامعه توسعه‌دهندگان بزرگ و پشتیبانی مداوم آن نیز باعث شده است این زبان به گزینه‌ای مطمئن برای توسعه سامانه‌های پژوهشی و نرم‌افزاری تبدیل شود. استفاده از پایتون در این پروژه علاوه بر تسهیل فرآیند توسعه، امکان دسترسی به مجموعه گسترده‌ای از ابزارها و کتابخانه‌ها را فراهم ساخته است که نقش مهمی در پیاده‌سازی قابلیت‌های سامانه داشته‌اند.

^۱ High-level

^۲ Guido Van Rossum

۲-۳-۲- رابط کاربری و ارائه نتایج

۲-۳-۲-۱- HTML^۱

این زبان پایه و اساس شکل‌گیری صفحات وب در فضای دیجیتال است و نخستین بار در اوایل دهه ۱۹۹۰ توسط تیم برنرز-لی^۲ معرفی شد. هدف اصلی این زبان، ارائه یک استاندارد برای ساختاردهی و نمایش اطلاعات در محیط وب بود. HTML در واقع یک زبان نشانه‌گذاری است که امکان تعریف و سازمان‌دهی عناصر صفحه مانند تیترها، پاراگراف‌ها، جداول و تصاویر را فراهم می‌کند. این ویژگی سبب شده است که این زبان نه تنها ابزاری برای توسعه وب، بلکه یک استاندارد جهانی برای ارائه داده‌ها محسوب شود [۲].

در پروژه حاضر، این زبان نقش ستون فقرات بخش گزارش خروجی را بر عهده دارد. تمامی محتوای تحلیلی، شامل نتایج آزمون‌ها، مسیرهای بحرانی، توصیف خطاها و جداول آماری، در قالب ساختارهای HTML پیاده‌سازی شده است. استفاده از آن این مزیت را ایجاد کرد که گزارش‌ها در هر مرورگری و روی هر سیستم‌عاملی بدون نیاز به نرم‌افزارهای جانبی خاص قابل مشاهده باشند. افزون بر این، به دلیل سادگی و انعطاف‌پذیری بالا، امکان یکپارچه‌سازی آسان با CSS و JavaScript را فراهم کرده و بدین ترتیب بستر اصلی تعاملات کاربر با خروجی سامانه شکل گرفته است.

۲-۳-۲-۲- CSS^۳

این زبان به‌عنوان مکمل HTML معرفی شد تا محدودیت‌های آن در زمینه‌ی طراحی و ظاهر صفحات برطرف گردد. HTML صرفاً قادر به تعریف ساختار محتوا بود، اما CSS توانست لایه‌ای مجزا برای مدیریت استایل و زیبایی‌شناسی صفحات وب فراهم آورد. این زبان امکان تغییر رنگ‌ها، فونت‌ها، اندازه‌ها، حاشیه‌ها و موقعیت عناصر مختلف را فراهم کرده و تجربه‌ی کاربری را به شکل چشمگیری ارتقا داده است [۳].

در سامانه حاضر، این زبان به‌عنوان ابزار اصلی برای بهبود خوانایی و زیبایی گزارش‌ها به کار گرفته شد. برای مثال، ردیف‌های مربوط به تست‌های موفق و ناموفق با رنگ‌های متفاوت نمایش داده می‌شوند تا در نگاه اول وضعیت کلی آزمون‌ها مشخص گردد. همچنین، CSS قابلیت طراحی واکنش‌گرا را در اختیار توسعه‌دهندگان قرار می‌دهد که بدین وسیله خروجی سامانه در دستگاه‌های مختلف- از مانیتورهای بزرگ گرفته تا گوشی‌های هوشمند- بدون به‌هم‌ریختگی ساختار نمایش داده می‌شود. بدین ترتیب، استفاده از این زبان تضمین می‌کند که گزارش تولیدی، علاوه بر صحت اطلاعات، از منظر زیبایی و تجربه کاربری نیز در سطح مطلوبی قرار گیرد.

^۱ HyperText Markup Language

^۲ Tim Berners-Lee

^۳ Cascading Style Sheets

۲-۳-۲- جاوااسکریپت

این زبان از مهم‌ترین زبان‌های برنامه‌نویسی پویا^۱ و شیء‌گرا^۲ در حوزه وب است که در ابتدا برای افزودن قابلیت‌های ساده به صفحات وب طراحی شد، اما امروزه به یک فناوری محوری در توسعه نرم‌افزارهای تحت وب تبدیل شده است. ویژگی اصلی جاوااسکریپت، اجرای کد در سمت کاربر^۳ و امکان به‌روزرسانی بخش‌هایی از صفحه بدون نیاز به بارگذاری مجدد کامل است. این زبان، پایه بسیاری از برنامه‌های تعاملی، چارچوب‌های فرانت‌اند^۴ مدرن و حتی برخی از فناوری‌های سمت سرور^۵ مانند Node.js را تشکیل داده است.

در این سامانه، زبان جاوااسکریپت نقش اساسی در ایجاد قابلیت‌های پویا و تعاملی گزارش‌ها بر عهده دارد. این زبان مسئولیت دارد داده‌های خروجی شامل اطلاعات پروژه، نتایج آزمون‌ها، خطاها و شاخص‌های کلیدی عملکرد را بارگذاری کرده و آن‌ها را به عناصر مختلف DOM^۶ در صفحه HTML متصل نماید. به کمک این مکانیزم، جداول مربوط به خطاها، نتایج آزمون و پیشنهادهای اصلاحی به صورت خودکار ساخته و به‌روزرسانی می‌شوند. افزون بر این، نمایش جزئیات خطاها و مکان‌های احتمالی وقوع آن‌ها به صورت چندلایه (نزدیک‌ترین به گراف، سرخ‌های متنی و پیشنهادهای مدل زبانی) در خروجی به کاربر ارائه می‌گردد. این سطح از پویایی باعث می‌شود گزارش نهایی تنها یک سند ایستا نباشد، بلکه محیطی تعاملی برای تحلیل و بررسی داده‌ها باشد؛ به گونه‌ای که کاربر می‌تواند با مشاهده و کاوش لایه‌های مختلف اطلاعات، درک عمیق‌تری از وضعیت سامانه و مسیرهای بحرانی به دست آورد.

۲-۳-۴- Tailwind CSS

چارچوبی مدرن در حوزه طراحی رابط کاربری^۷ است که در سال ۲۰۱۷ معرفی شد و به سرعت توانست جایگاه ویژه‌ای در میان توسعه‌دهندگان وب پیدا کند. تفاوت اصلی آن با چارچوب‌های سنتی مانند Bootstrap در فلسفه طراحی آن نهفته است؛ جایی که Tailwind رویکردی موسوم به Utility-First را دنبال می‌کند. در این رویکرد، به جای استفاده از کامپوننت‌های از پیش ساخته‌شده یا نیاز به تعریف فایل‌های CSS سفارشی بزرگ، مجموعه‌ای گسترده از کلاس‌های کوچک و هدفمند ارائه می‌شود که توسعه‌دهندگان می‌توانند آن‌ها را مانند آجرهای Lego در کنار هم قرار دهند و به سرعت استایل‌های پیچیده و متنوعی ایجاد کنند. از ویژگی‌های کلیدی Tailwind می‌توان به انعطاف‌پذیری بالا، پشتیبانی کامل از طراحی واکنش‌گرا،

^۱ Dynamic

^۲ Object-oriented

^۳ Client-Side

^۴ Front-end

^۵ Server-Side

^۶ Document Object Model

^۷ User Interface

سفارشی‌سازی ساده از طریق فایل پیکربندی و سازگاری با فریم‌ورک‌های جاوااسکریپتی مانند React و Vue اشاره کرد. این چارچوب علاوه بر کاهش حجم کدنویسی CSS، باعث افزایش سرعت توسعه و ارتقای کیفیت کد می‌شود.

در این سامانه، استفاده از Tailwind CSS دو مزیت عمده داشت: نخست، افزایش سرعت توسعه به دلیل وجود کلاس‌های آماده و استاندارد. دوم، ایجاد هماهنگی و یکپارچگی در طراحی که باعث شد تمامی اجزای خروجی - شامل دکمه‌ها، فرم‌ها، جداول و حتی بخش‌های تحلیلی مانند گراف‌ها - ظاهری منسجم و سازگار پیدا کنند. همچنین، پشتیبانی داخلی از طراحی واکنش‌گرا^۱ تضمین کرد که گزارش خروجی روی انواع دستگاه‌ها از موبایل تا دسکتاپ بدون مشکل و با کیفیتی یکنواخت نمایش داده شود.

۲-۳-۵- کتابخانه‌های مصورسازی داده

Chart.js یک کتابخانه متن‌باز^۲ جاوااسکریپتی است که نخستین بار در سال ۲۰۱۳ منتشر شد و از آن زمان تاکنون به یکی از محبوب‌ترین ابزارهای مصورسازی داده در وب تبدیل شده است. دلیل این محبوبیت، سادگی در استفاده، حجم سبک، و ارائه نمودارهای گرافیکی زیبا و تعاملی است. این کتابخانه طیف گسترده‌ای از نمودارها شامل خطی، میله‌ای، دایره‌ای^۳، دونات^۴، رادار، حبابی و پراکندگی را پشتیبانی می‌کند و امکان شخصی‌سازی گسترده در رنگ‌ها، فونت‌ها، برجسب‌ها و انیمیشن‌ها را برای توسعه‌دهنده فراهم می‌سازد. از دیگر نقاط قوت Chart.js می‌توان به پشتیبانی از تعامل کاربر اشاره کرد؛ به گونه‌ای که با حرکت ماوس بر روی هر بخش از نمودار یا کلیک روی آن، جزئیات بیشتری نمایش داده می‌شود. این قابلیت باعث می‌شود نمودارها نه صرفاً جنبه تزئینی داشته باشند، بلکه به عنوان ابزارهای واقعی برای تحلیل و تصمیم‌گیری مورد استفاده قرار گیرند.

در این سامانه، Chart.js برای نمایش وضعیت اجرای تست‌ها، درصد موفقیت و شکست آزمون‌ها، و تحلیل مسیرهای بحرانی به کار گرفته شد. به کمک آن، کاربران قادر بودند الگوها و نقاط ضعف سیستم را در کوتاه‌ترین زمان ممکن شناسایی کنند. افزون بر این، ماهیت پویا و تعاملی نمودارها تجربه‌ی کاربری غنی‌تری ایجاد کرد و خروجی گزارش را از یک فایل ایستا به محیطی تعاملی و تحلیلی ارتقا داد.

^۱ Responsive

^۲ Open Source

^۳ Pie

^۴ Doughnut

۲-۳-۳- ابزارهای خط فرمان^۱

یکی از قدیمی‌ترین و در عین حال پرکاربردترین ابزارهای تعامل انسان و رایانه محسوب می‌شوند. در این مدل از تعامل، کاربر به جای استفاده از رابط‌های گرافیکی پیچیده، از طریق نوشتن دستورات متنی با سامانه ارتباط برقرار می‌کند. این شیوه اگرچه ممکن است برای کاربران عادی اندکی دشوار به نظر برسد، اما در میان برنامه‌نویسان، توسعه‌دهندگان و مدیران سیستم‌ها به دلیل سرعت، انعطاف‌پذیری و قابلیت اسکریپت‌نویسی جایگاه ویژه‌ای دارد.

این ابراز از همان دهه‌های نخستین ظهور سیستم‌های عامل نظیر یونیکس^۲ و داس^۳ شکل گرفت و امروزه همچنان در کنار محیط‌های گرافیکی به‌عنوان ابزاری قدرتمند مورد استفاده قرار می‌گیرد. علت ماندگاری و اهمیت آن را می‌توان در دو ویژگی کلیدی دانست: نخست، کنترل مستقیم و دقیق بر روی عملکرد سامانه، و دوم، امکان اتوماسیون و تکرارپذیری وظایف از طریق اسکریپت‌ها و فایل‌های پیکربندی. همین ویژگی‌ها باعث شده است که این ابراز نه‌تنها از میان نرود، بلکه در حوزه‌هایی مانند توسعه نرم‌افزارهای سیستمی، ابزارهای برنامه‌نویسی و مدیریت سرورها بیش از پیش مورد توجه قرار گیرد.

در سامانه حاضر، طراحی و پیاده‌سازی CLI نقش مهمی در تحقق اهداف پروژه داشته است. این سامانه با هدف اصلی تحلیل کد، پردازش لاگ‌های تست و تولید گزارش‌های تحلیلی برای برنامه‌نویسان و تسترها طراحی شده است. از آنجا که مخاطبان اصلی پروژه کاربران حرفه‌ای و فنی هستند، ارائه‌ی یک رابط گرافیکی تحت وب که نیازمند راه‌اندازی سرور و تبادل کدهای محرمانه باشد، نه تنها امنیت را کاهش می‌داد بلکه کار با سامانه را پیچیده‌تر می‌کرد. در نتیجه، تصمیم به توسعه یک اپلیکیشن خط فرمانی سبک و مستقل گرفته شد.

این اپلیکیشن با زبان پایتون توسعه یافت و از کتابخانه‌های استاندارد مانند argparse برای مدیریت ورودی‌ها و آرگومان‌ها استفاده کرد. کاربر می‌تواند با اجرای یک دستور ساده، مسیر پروژه، نوع زبان برنامه‌نویسی، فایل‌های لاگ و همچنین تنظیمات خروجی را مشخص کند. سپس CLI تمامی مراحل از ساخت گراف وابستگی، پردازش نتایج آزمون‌ها، استخراج مسیرهای بحرانی، تولید پرامپت برای مدل زبانی و در نهایت تولید گزارش HTML را به‌صورت خودکار انجام می‌دهد.

از منظر معماری نرم‌افزار، CLI مزیت‌های متعددی را در این پروژه فراهم کرد. نخست، عدم نیاز به سرور مرکزی موجب شد تا فرآیند تحلیل به‌طور کامل روی سیستم شخصی کاربر انجام گیرد و نگرانی‌های امنیتی درباره‌ی اشتراک‌گذاری کدهای منبع از بین برود. همچنین، قابلیت پشتیبانی از چند حالت اجرا شامل حالت

^۱ Command Line Interface

^۲ Unix

^۳ DOS

کامل^۱ و حالت سریع^۲ را دارد که به کاربر اجازه می‌دهد بسته به نیاز خود، از اجرای مجدد تمامی مراحل یا استفاده از داده‌های کش‌شده بهره‌گیرد.

در نهایت، استفاده از CLI به‌عنوان رابط اصلی این سامانه باعث شد تا خروجی پروژه ساده، سریع، ایمن و سازگار با نیازهای جامعه‌ی هدف باشد. این انتخاب نشان داد که برخلاف تصور عمومی مبنی بر ضرورت وجود رابط‌های گرافیکی، در پروژه‌های تخصصی و فنی، این ابراز همچنان یک انتخاب بهینه و منطقی است.

۲-۴-۲- مدل‌های زبانی بزرگ

۲-۴-۲-۱- معرفی و مبانی نظری

مدل‌های زبانی بزرگ با اتکا به معماری ترنسفورمر و آموزش در مقیاس بالا، قادرند الگوهای معنایی و نحوی را از متون متنوع فراگیرند و در وظایف مختلف زبانی به‌کار گیرند. در این پروژه، نقش مدل زبانی، «تحلیل‌گر ثانویه» است که بر مبنای شواهد ساختاری (گراف و مسیرها) و شواهد اجرایی (گزارش‌های آزمون)، روایت منسجمی از ریشه خطا و راه‌حل‌های قابل‌اقدام ارائه می‌دهد. برای بهره‌وری بیشتر، ورودی مدل به‌صورت ساختاریافته آماده می‌شود تا بار تفسیر کاهش یابد و تمرکز مدل بر استدلال و جمع‌بندی قرار گیرد.

۲-۴-۲-۲- روش‌های تعامل و به‌کارگیری

تعامل با مدل در چارچوب پرامپت‌های ساختاریافته انجام می‌شود. نمونه‌های راهنما، طرح JSON^۳ خروجی، خلاصه‌ی آزمون‌ها و مسیرهای منتخب به‌صورت کنترل‌شده در اختیار مدل قرار می‌گیرد. این رویکرد، رفتار مدل را پایدارتر می‌کند و خروجی‌ها را برای مصرف ماشینی مناسب می‌سازد. در جایی که لازم است، از few-shot examples برای شکل‌دهی سبک پاسخ‌ها و مرزبندی فرمت استفاده می‌شود تا مدل از ارائه متن آزاد بدون ساختار خودداری کند.

۲-۴-۳- کاربردها در تحلیل و پردازش داده

کاربرد اصلی مدل در این سامانه، تبیین ریشه‌ی شکست تست‌ها، گروه‌بندی الگوهای خطا و پیشنهاد اصلاحات عملی است. مدل با اتکا به توصیف مسیرهای دخیل و پیام‌های خطا، میان علت‌های محتمل تمایز می‌گذارد و پیشنهادها را در قالب قابل اجرا برای تیم توسعه باز می‌گرداند. خروجی نهایی به‌گونه‌ای طراحی شده که هم برای مصورسازی در گزارش مناسب باشد و هم برای اتوماسیون‌های بعدی قابل پردازش باقی بماند.

^۱ Full Mode

^۲ Quick Mode

^۳ JavaScript Object Notation

۲-۵- گراف و پردازش داده‌های آزمون

۲-۵-۱- مفاهیم گراف وابستگی و ساختار آن در سامانه

گراف وابستگی، بازنمایی ساختاری روابط درون سامانه‌ی نرم‌افزاری است که در آن هر گره نمایانگر یک واحد رفتاری مانند تابع، متد یا تست است و هر یال نشان‌دهنده‌ی یک رابطه‌ی فراخوانی یا اتکا. هویت گره‌ها بر اساس ترکیبی از مسیر فایل و نام واحد رفتاری تعریف می‌شود تا هر عنصر، شناسه‌ای یکتا و پایدار داشته باشد. این بازنمایی، امکان مشاهده‌ی جامع مسیرهای اجرای محتمل در تست‌ها و نقاط تمرکز وابستگی‌ها را فراهم می‌کند و بستری برای تحلیل علی شکست‌ها به‌وجود می‌آورد.

۲-۵-۲- نقش گراف در نمایش روابط بین اجزای نرم‌افزار

کارکرد گراف صرفاً نمایش روابط نیست، بلکه ابزار تحلیل است. با برچسب‌گذاری گره‌های متناظر با تست‌ها و اتصال آن‌ها به توابع و مسیرهای فراخوانی، می‌توان مسیرهایی را که با شکست‌ها هم‌زمانی دارند شناسایی کرد. گره‌هایی که در مسیرهای مرتبط با چند شکست تکرار می‌شوند، یا گره‌هایی با درجه‌ی اتکا بالا، به‌عنوان نقاط پرریسک شناخته می‌شوند. این شناخت، اولویت‌بندی اقدامات اصلاحی را هدایت می‌کند و به تیم توسعه کمک می‌کند با کمترین تغییر، بیشترین تأثیر را کسب کند.

۲-۵-۳- مفاهیم استانداردسازی داده‌های آزمون و نقش آن در تحلیل

گزارش‌های آزمون بسته به چارچوب اجرا، فرمت‌های متفاوتی دارند، اما برای تحلیل یکپارچه لازم است به قالب یکنواخت تبدیل شوند. استانداردسازی، اطلاعات کلیدی مانند نام تست، وضعیت اجرا و پیام خطا را به فیلدهای هم‌معنا تبدیل می‌کند و نویزهای قالبی را حذف می‌نماید. این یکنواختی دو اثر مهم دارد: اول، نگاشت نام تست‌ها به شناسه‌های گراف ساده و قابل اعتماد می‌شود. دوم، تحلیل‌های آماری و روندی بر پایه شمارش‌ها و نوع خطاها قابل اتکا می‌گردد. بدون استانداردسازی، هر تحلیل مقایسه‌ای میان اجراها یا میان زبان‌ها با خطا و ابهام همراه خواهد بود.

۲-۵-۴- نگاشت مفهومی داده‌های آزمون بر روی گراف و اهمیت آن در تصمیم‌گیری

پس از استانداردسازی، نگاشت مفهومی میان نتایج آزمون و گراف شکل می‌گیرد. هدف، نسبت دادن هر رکورد آزمون به گره متناظر در گراف و رنگ‌آمیزی مسیرهای مرتبط با شکست‌ها است. نگاشت موفق، سه دستاورد دارد: نخست، گزارش تعاملی می‌تواند وضعیت تست‌ها را در بافت کد نمایش دهد؛ دوم، تحلیل

مسیرهای بحرانی امکان‌پذیر می‌شود و می‌توان روابطی مانند «این شکست‌ها بیشتر از این مسیر می‌گذرند» را مستند کرد؛ سوم، ورودی مدل زبانی با شواهد ساختاری دقیق تغذیه می‌شود و کیفیت تبیین علل و پیشنهادها افزایش می‌یابد. بنابراین نگاشت، حلقه‌ی اتصال شواهد اجرایی به شواهد ساختاری است و بنیان تصمیم‌گیری مهندسی در این سامانه را تشکیل می‌دهد.

۲-۶- چارچوب و سرویس‌های پشتیبان

۲-۶-۱- مدیریت نسخه و انتشار پروژه

گیت^۱ کی از پرکاربردترین سامانه‌های کنترل نسخه^۲ در حوزه توسعه نرم‌افزار است که در سال ۲۰۰۵ توسط لینوس توروالدز^۳ ایجاد شد. این ابزار بر اساس معماری توزیع‌شده^۴ عمل می‌کند، به این معنا که هر توسعه‌دهنده یک نسخه کامل از تاریخچه پروژه را روی سیستم خود دارد و مستقل از مخزن اصلی قادر به ایجاد تغییرات و مدیریت شاخه‌ها است. چنین ساختاری انعطاف‌پذیری بالایی در مدیریت تیمی پروژه و امکان بازگشت به نسخه‌های قبلی را فراهم می‌سازد.

از ویژگی‌های کلیدی ابزار می‌توان به امکان شاخه‌بندی^۵ و ادغام^۶ سریع، ردیابی دقیق تغییرات فایل‌ها، پشتیبانی گسترده در سیستم‌عامل‌های مختلف و هم‌افزایی با سرویس‌های میزبانی مانند GitHub و GitLab اشاره کرد. این قابلیت‌ها، فرآیند توسعه نرم‌افزار را نظام‌مندتر، امن‌تر و قابل اعتمادتر می‌سازد.

در این سامانه، استفاده از گیت برای مدیریت کد منبع و کنترل تغییرات نقشی حیاتی ایفا کرده است. توسعه‌دهنده می‌تواند با ایجاد شاخه‌های مستقل، ویژگی‌های جدید را آزمایش کرده و پس از بررسی و تست، تغییرات را با اطمینان به شاخه اصلی منتقل کند. همچنین، تاریخچه کامل پروژه امکان ردیابی خطاها و بازگشت به نسخه‌های پایدار را فراهم می‌سازد. به این ترتیب، گیت نه تنها ابزاری برای نگهداری کد بلکه بستری برای تضمین کیفیت، پایداری و همکاری در توسعه پروژه بوده است.

۲-۶-۲- انتشار بسته نرم‌افزاری در Poetry با PyPI

یکی از مراحل مهم در چرخه توسعه نرم‌افزار، انتشار^۷ پروژه برای استفاده عمومی یا سازمانی است. در اکوسیستم پایتون، اصلی‌ترین مخزن رسمی برای توزیع بسته‌ها، PyPI^۸ است. انتشار پروژه در PyPI این امکان

^۱ Git

^۲ Version Control System

^۳ Linus Torvalds

^۴ Distributed

^۵ Branching

^۶ Merging

^۷ Publish

^۸ Python Package Index

را فراهم می‌سازد که سایر توسعه‌دهندگان بتوانند تنها با یک دستور ساده مانند `pip install`، پروژه را روی سیستم خود نصب و اجرا کنند.

برای مدیریت این فرآیند در سامانه حاضر، از Poetry استفاده شد. این ابزاری مدرن برای مدیریت وابستگی‌ها، ساخت^۱ بسته، و انتشار در PyPI است که نسبت به ابزارهای قدیمی‌تر مانند `setuptools` یا `pip` مزایای متعددی دارد. Poetry یک فایل پیکربندی استاندارد با فرمت `pyproject.toml` ایجاد می‌کند که در آن تمامی اطلاعات پروژه، نسخه‌ها، وابستگی‌ها و فراداده‌ها مشخص شده و به شکل یکپارچه مدیریت می‌شود. استفاده از Poetry در این پروژه چند مزیت کلیدی داشت: نخست، مدیریت ساده و دقیق وابستگی‌ها که مانع از بروز ناسازگاری میان کتابخانه‌های مختلف شد. دوم، فرآیند خودکار ساخت و انتشار که تنها با چند دستور ساده امکان‌پذیر شد (`poetry build` و `poetry publish`). سوم، افزایش قابلیت بازتولید^۲، چرا که Poetry نسخه دقیق کتابخانه‌ها را در فایل قفل^۳ ذخیره کرده و در هر محیطی همان شرایط را بازسازی می‌کند. در آخر انتشار پروژه در PyPI باعث شد که سامانه حاضر به راحتی در دسترس سایر پژوهشگران، توسعه‌دهندگان و تست‌کنندگان نرم‌افزار قرار گیرد و تنها با یک دستور، بر روی سیستم‌های مختلف نصب شود. این قابلیت، پروژه را از یک نمونه آزمایشی به یک ابزار قابل اعتماد و استاندارد برای استفاده عملی ارتقا داد.

۲-۶-۳- سرویس‌های خارجی

تعامل با مدل زبانی از طریق یک API^۴ خارجی انجام می‌شود. داده‌های ساختاریافته شامل خلاصه‌ی وضعیت آزمون‌ها، شناسه‌ی مسیرهای منتخب و برش‌های کد، به سرویس ارسال می‌شود و پاسخ در قالب قابل مصرف ماشینی بازگردانده می‌شود. این لایه‌ی اتصال طوری طراحی شده که با رعایت اصول حداقل‌گرایی در داده، محرمانگی پروژه‌ی کاربر حفظ شود و هیچ‌گونه نگهداری غیرضروری از محتوای کد صورت نگیرد.

۲-۶-۳-۱- سرویس AvalAI

سرویس AvalAI نقش موتور استدلال زبانی را ایفا می‌کند. راه‌اندازی به صورت کلیدمحرور انجام می‌شود و سیاست‌های زمانی و حد نرخ فراخوانی قابل کنترل هستند. با توجه به ساختارمند بودن پرامپت‌ها و خروجی‌ها، پاسخ‌های مدل با حداقل پردازش در سامانه ادغام می‌شوند و برای ارائه گزارش یا اقدامات بعدی آماده‌اند.

^۱ Build

^۲ Reproducibility

^۳ Lock file

^۴ Application Programming Interface

۷-۲- جمع‌بندی

در این فصل، معماری سامانه و فناوری‌های اصلی آن معرفی شد. هسته پردازشی با پایتون توسعه یافت و نتایج با HTML، CSS و جاوااسکریپت در قالب محیطی تعاملی ارائه شدند. برای انسجام و واکنش‌گرایی رابط، از Tailwind CSS بهره گرفته شد و Chart.js امکان مصورسازی پویا و تحلیلی داده‌ها را فراهم ساخت. بخش گراف با استخراج وابستگی‌ها میان اجزای کد و آزمون‌ها، بستری برای شناسایی نقاط بحرانی و تحلیل عمیق‌تر کیفیت سامانه ایجاد کرد. همچنین، استفاده از Git برای مدیریت نسخه‌ها و Poetry برای مدیریت وابستگی‌ها و انتشار پروژه در PyPI، توسعه و نگهداشت سامانه را ساده و مقیاس‌پذیر ساخت. بطور کلی، ترکیب این فناوری‌ها و ابزارها سامانه‌ای یکپارچه، قابل توسعه و پژوهش‌محور پدید آورد که علاوه بر ارائه گزارش‌های دقیق و تعاملی، زمینه‌ساز توسعه‌های آتی نیز خواهد بود.

فصل سوم

پیشینه پژوهشی

۳-۱- مقدمه

در سال‌های اخیر، موضوع تحلیل و گزارش‌دهی نتایج تست نرم‌افزار به یکی از بخش‌های مهم در چرخه توسعه سیستم‌ها تبدیل شده است. با گسترش ابزارهای خودکار و رویکردهای نوین مانند هوش مصنوعی و مدل‌های زبانی بزرگ، پژوهش‌های متعددی در زمینه بهبود کیفیت تست، شناسایی خطاها و افزایش دقت در تحلیل نتایج انجام گرفته است. هر یک از این تحقیقات تلاش کرده‌اند تا با معرفی ابزارها و روش‌های نو، روند شناسایی و ریشه‌یابی خطاها را ساده‌تر و سریع‌تر کنند. با این حال، هنوز چالش‌هایی همچون پراکندگی داده‌ها، ضعف در ارائه گزارش‌های یکپارچه و نیاز به تحلیل عمیق‌تر باقی مانده است. در این فصل، با مروری بر کارهای مرتبط و بررسی ابزارهای مطرح، تلاش می‌شود تصویری روشن از دستاوردهای پیشین و جایگاه پژوهش حاضر ارائه گردد.

۳-۲- معرفی کارهای مرتبط

۳-۲-۱- مطالعه و بررسی مقاله مرتبط

یکی از مقالات مهم مورد بررسی در این پژوهش به تحلیل توانایی مدل‌های زبانی بزرگ در تولید تست‌کیس‌های خودکار اختصاص دارد. تولید تست‌کیس یکی از بخش‌های حیاتی در چرخه توسعه نرم‌افزار است، زیرا امکان بررسی صحت عملکرد کد در شرایط متنوع و پوشش بخش‌های مختلف آن را فراهم می‌سازد. با این حال، این فرآیند معمولاً پرهزینه و زمان‌بر است. نویسندگان مقاله با هدف رفع این چالش، چارچوبی جامع با نام TESTEVAL ارائه کرده‌اند که بر اساس ۲۱۰ برنامه منتخب از پلتفرم LeetCode و معیار پیچیدگی سیکلوماتیک طراحی شده است. این چارچوب به گونه‌ای ساخته شده که سنجش توانایی مدل‌ها در پوشش خطوط، شاخه‌ها و مسیرهای پیچیده کد را امکان‌پذیر می‌سازد و بدین ترتیب بستری نظام‌مند برای مقایسه و

ارزیابی مدل‌ها فراهم می‌آورد [۴].

مقاله مذکور سه وظیفه اصلی را برای ارزیابی مدل‌ها تعریف می‌کند: پوشش کلی کد^۱، پوشش هدفمند خطوط یا شاخه‌های مشخص^۲ و پوشش مسیرهای خاص اجرایی^۳. برای هر کدام از این وظایف، عملکرد مدل‌ها از نظر صحت نحوی، قابلیت اجرا و درستی عبارات Assertion مورد ارزیابی قرار گرفته است. همچنین شاخص نوینی با عنوان cov@K معرفی شده که میزان تنوع تست کیس‌های تولیدشده را با توجه به تعداد انتخاب‌های محدود K اندازه‌گیری می‌کند. استفاده از این رویکردها نشان داد که اگرچه مدل‌هایی نظیر GPT-4 توانسته‌اند به دقتی بیش از ۹۸٪ در پوشش خطوط و ۹۷٪ در پوشش شاخه‌ها برسند، اما همچنان در تولید تست‌های هدفمند و تحلیل مسیرهای منطقی پیچیده ضعف دارند [۴].

نتایج تجربی مقاله نشان داد که مدل‌های تجاری مانند GPT-4 و Gemini در مقایسه با مدل‌های متن‌باز نظیر DeepSeek و CodeLlama عملکرد بهتری دارند، اما در وظایف مرتبط با مسیرهای خاص و استدلال منطقی همچنان محدودیت‌های جدی وجود دارد. علاوه بر این، تکنیک‌های کمکی مانند COT^۴ نیز تنها در برخی مدل‌ها توانستند بهبود نسبی ایجاد کنند. جمع‌بندی نویسندگان مقاله حاکی از آن است که اگرچه LLM ها در تولید تست‌های متنوع و گسترده عملکرد چشمگیری دارند، اما برای بهره‌گیری عملی از آن‌ها در پروژه‌های واقعی نرم‌افزاری لازم است روش‌های ترکیبی و هوشمندتر توسعه یابند.

مطالعه این مقاله نقطه عطفی برای آغاز پژوهش حاضر بود. با بررسی دقیق روش‌های ارائه‌شده در چارچوب TESTEVAL، به‌ویژه استفاده از مدل‌های زبانی بزرگ و تکنیک‌های پیشرفته‌ای همچون COT در تولید و ارزیابی تست کیس‌ها، مسیر طراحی پروژه روشن شد. بر همین اساس، تصمیم گرفته شد سامانه‌ای پیاده‌سازی گردد که تمرکز آن بر تحلیل نتایج تست‌های نرم‌افزاری به‌صورت داده‌محور و تحلیلی باشد. در این پروژه نیز مشابه مقاله، علاوه بر تولید گزارش‌ها و پوشش‌دهی تست‌ها، روش‌های تحلیلی برای شناسایی الگوهای خطا، ارزیابی تنوع تست‌ها و سنجش کیفیت خروجی‌ها به کار گرفته شده است. بدین ترتیب، مقاله مذکور نه تنها مبنای نظری بلکه الهام‌بخش انتخاب روش‌ها و طراحی فرآیندهای عملی پژوهش حاضر بوده است.

۳-۲-۲- روش‌های تحلیل و پایش نتایج تست‌های نرم‌افزاری

در فرآیند توسعه نرم‌افزار، شکست در اجرای تست‌ها امری اجتناب‌ناپذیر است و شناسایی علل آن مستلزم رویکردی جامع و چندلایه است. مجموعه‌ای از روش‌های تحلیلی در این حوزه مطرح شده‌اند که ترکیبی از عیب‌یابی^۵، تحلیل داده، بازبینی کد و ارزیابی محیط اجرا را در بر می‌گیرند. یکی از مهم‌ترین این روش‌ها تحلیل

^۱ Overall Coverage

^۲ Targeted Line/Branch Coverage

^۳ Targeted Path Coverage

^۴ Chain-of- Thought

^۵ Debugging

لاگ‌ها و خروجی‌ها است که از طریق بررسی پیام‌های خطا و Stack Trace ها محل بروز مشکل شناسایی می‌شود. ابزارهایی مانند ELK Stack و Splunk با کشف الگوهای تکرارشونده در لاگ‌ها، ریشه‌ی خطاهایی همچون شکست‌های مکرر timeout را آشکار می‌سازند. همچنین گزارش‌های تولیدشده توسط چارچوب‌های تستی نظیر JUnit، TestNG و Pytest امکان ردیابی دقیق تست‌های شکست‌خورده و عبارات ناموفق را فراهم می‌کنند. در کنار این موارد، بازبینی کد^۱ و اجرای ایزوله تست‌ها روشی کارآمد برای حذف اثرات جانبی ناشی از سایر تست‌ها محسوب می‌شود و وابستگی‌های ناخواسته یا خطاهای ناشی از وضعیت‌های مشترک را مشخص می‌سازد.

از سوی دیگر، تحلیل محیط اجرای تست در شناسایی ناسازگاری‌ها اهمیت ویژه‌ای دارد، چراکه تنظیمات نادرست، نسخه‌های ناسازگار وابستگی‌ها یا محدودیت منابع سیستم می‌توانند عامل شکست باشند. تحلیل طراحی و منطق تست‌ها نیز نقش مهمی در ارزیابی صحت Assertions، کیفیت داده‌های ورودی و استقلال تست‌ها ایفا می‌کند و مشکلاتی نظیر شکست‌های مقطعی ناشی از رفتار غیرقطعی (مانند شرایط رقابتی یا ورودی‌های تصادفی) را نمایان می‌سازد. علاوه بر این، تحلیل مقایسه‌ای^۲ با مقایسه نتایج اجرای جدید با نسخه‌های پیشین، تغییرات منجر به شکست را مشخص کرده و ابزارهای کنترلی مانند Git blame و bisect Git در این مسیر مورد استفاده قرار می‌گیرند. در نهایت، تحلیل ریشه‌ای^۳ با روش‌هایی چون 5 Whys و Fault Tree Analysis دلایل بنیادی شکست‌ها را آشکار ساخته و امکان پیشگیری از تکرار آن‌ها را فراهم می‌کند. این مجموعه روش‌ها چارچوبی نظام‌مند برای پایش و تحلیل نتایج تست‌ها ایجاد کرده و در پژوهش حاضر به‌عنوان مبنای ارزیابی ابزارهای منتخب مورد استفاده قرار گرفته است.

۳-۳- معرفی و ارزیابی ابزارهای منتخب

۳-۳-۱- TestComplete

در حال حاضر، TestComplete یکی از پرکاربردترین ابزارهای تجاری تست نرم‌افزار به شمار می‌رود و در بسیاری از سازمان‌های بزرگ فناوری و صنایع مالی به‌عنوان ابزار استاندارد تست رابط کاربری^۴، تست‌های رگرسیون^۵ و تست بار^۶ استفاده می‌شود. این ابزار از زبان‌هایی مانند JavaScript، Python، C++Script و VBScript پشتیبانی می‌کند و امکان اجرای تست در محیط‌های دسکتاپ، وب و موبایل را فراهم می‌سازد. قابلیت‌های کلیدی همچون شناسایی هوشمند عناصر رابط کاربری، ضبط و بازپخش سناریوها، و یکپارچگی با

^۱ Code Inspection

^۲ Comparative Analysis

^۳ Root Cause Analysis

^۴ UI Testing

^۵ Regression Testing

^۶ Load Testing

ابزارهای مدیریت نسخه و خطوط CI/CD، جایگاه این ابزار را در فرآیندهای توسعه مدرن تقویت کرده است. از نظر مزایا و معایب، TestComplete به دلیل پوشش فناوری گسترده، سهولت در ایجاد تست‌ها بدون نیاز به کدنویسی زیاد و گزارش‌های تصویری جامع، به‌عنوان یکی از قدرتمندترین ابزارهای موجود شناخته می‌شود. پشتیبانی مستمر SmartBear و اکوسیستم گسترده آن نیز باعث شده است که این ابزار به‌طور منظم به‌روزرسانی شود و با فناوری‌های جدید سازگار باقی بماند. با این حال، هزینه‌های بالای لایسنس و وابستگی به سیستم‌عامل ویندوز از مهم‌ترین محدودیت‌های آن هستند. همچنین به دلیل گستردگی امکانات، یادگیری کامل ابزار ممکن است برای تیم‌های تازه‌کار زمان‌بر باشد. در مجموع، ابزاری کارآمد برای سازمان‌های بزرگ و پروژه‌های میان‌پلتفرمی است که با وجود چالش‌های هزینه‌ای و پیچیدگی، ارزش افزوده‌ی بالایی در تضمین کیفیت نرم‌افزار ایجاد می‌کند.

۳-۲-۳ TestRail

یک پلتفرم تحت وب برای مدیریت چرخه عمر تست است که نخستین بار توسط شرکت آلمانی Gurock Software توسعه یافت و بعدها در اختیار شرکت Idera قرار گرفت. ایده‌ی اصلی طراحی این ابزار پاسخ به نیاز تیم‌های نرم‌افزاری برای سازماندهی سیستماتیک تست‌ها و ایجاد هماهنگی بین تیم‌های توسعه و تضمین کیفیت بود. در مقیاس پروژه‌های بزرگ، ثبت و پیگیری تست‌ها به‌صورت دستی یا پراکنده باعث بروز خطاهای مدیریتی و عدم شفافیت در روند توسعه می‌شد. این ابزار با ارائه یک محیط متمرکز تحت وب این امکان را فراهم ساخت که تست‌کیس‌ها به‌طور ساختارمند تعریف شوند و چرخه‌ی کامل تست شامل طراحی، زمان‌بندی، اجرا و ارزیابی نتایج در یک بستر واحد مدیریت گردد [۵].

امروزه TestRail به‌عنوان یکی از پرکاربردترین ابزارهای مدیریت تست در صنایع مختلف، به‌ویژه در حوزه‌های مالی، مخابرات و شرکت‌های بزرگ فناوری شناخته می‌شود. این ابزار برخلاف ابزارهای صرفاً گزارش‌دهی، تمرکز اصلی خود را بر مدیریت و ردیابی تست‌ها قرار داده و با ارائه‌ی داشبوردهای تحلیلی، وضعیت کیفی نرم‌افزار را از منظر مدیریتی نمایش می‌دهد. از نقاط قوت مهم TestRail می‌توان به یکپارچگی با سیستم‌های مدیریت پروژه مانند Jira، Azure DevOps و GitHub و همچنین پشتیبانی از خطوط CI/CD اشاره کرد. این یکپارچگی امکان ردگیری تست‌ها را از سطح کد تا سطح مدیریت پروژه فراهم می‌سازد و به تیم‌ها کمک می‌کند ارتباطی شفاف میان نتایج تست و وظایف توسعه برقرار کنند [۵].

از منظر مزایا و معایب، TestRail ابزاری قدرتمند در زمینه نظم‌دهی و مدیریت فرآیند تست به‌شمار می‌رود و با ارائه داشبوردهای مدیریتی و قابلیت‌های ردیابی جامع، به تیم‌ها کمک می‌کند درک دقیقی از وضعیت پروژه داشته باشند. همچنین، پشتیبانی از پروژه‌های چند تیمی و تسهیل همکاری میان تسترها و توسعه‌دهندگان یکی از ارزش‌های کلیدی آن است. اما در کنار این مزایا، باید توجه داشت که TestRail برخلاف ابزارهایی نظیر ReportPortal یا Allure فاقد قابلیت اجرای مستقیم تست یا تحلیل خودکار خطاهاست و عملاً تنها به‌عنوان

یک سکوی مدیریتی عمل می‌کند. این موضوع باعث می‌شود برای تکمیل زنجیره آزمون نیازمند اتصال به ابزارهای جانبی باشد. در نتیجه، می‌توان گفت TestRail برای سازمان‌هایی که به دنبال مدیریت متمرکز و تحلیلی چرخه تست هستند بسیار سودمند است، اما برای تیم‌هایی که انتظار تحلیل تعاملی و عمیق از نتایج تست دارند، به تنهایی پاسخگو نخواهد بود.

۳-۳-۳ ReportPortal

یک ابزار متن‌باز و مدرن برای مدیریت و تحلیل نتایج تست‌های نرم‌افزاری است که توسط جامعه‌ای از توسعه‌دهندگان تحت حمایت سازمان EPAM^۱ معرفی شد. ایده اصلی پشت ReportPortal پاسخ به محدودیت‌های ابزارهای سنتی گزارش‌دهی بود که صرفاً داده‌ها را ثبت می‌کردند اما توانایی تحلیل هوشمند نداشتند. با رشد تیم‌های DevOps و گسترش CI/CD در پروژه‌های نرم‌افزاری، نیاز به پلتفرمی که بتواند در لحظه نتایج تست‌ها را جمع‌آوری، دسته‌بندی و تحلیل کند، بیش از پیش احساس شد. ReportPortal با معماری مبتنی بر میکروسرویس‌ها و پشتیبانی از استقرار در Docker و Kubernetes به‌گونه‌ای طراحی شد که به راحتی با فرآیندهای مدرن توسعه و استقرار سازگار شود [۶].

امروزه ReportPortal به‌طور گسترده در سازمان‌هایی که حجم بالایی از تست‌های خودکار اجرا می‌کنند استفاده می‌شود، به‌ویژه در صنایع مالی، مخابرات و تجارت الکترونیک که کیفیت نرم‌افزار و تحلیل سریع شکست‌ها اهمیت حیاتی دارد. این ابزار قابلیت تحلیل ریشه‌ای^۲ مبتنی بر یادگیری ماشین را ارائه می‌دهد که می‌تواند شکست‌ها را به‌صورت خودکار دسته‌بندی کرده و الگوهای تکرارشونده را شناسایی کند. علاوه بر این، امکاناتی همچون Launch Comparison برای مقایسه چند اجرای متوالی تست و داشبوردهای پویا با نمودارهای تعاملی باعث شده است ReportPortal به عنوان یکی از پیشرفته‌ترین ابزارهای متن‌باز در حوزه تحلیل تست شناخته شود. جامعه فعال توسعه‌دهندگان و کاربران نیز با افزودن پلاگین‌ها و ارائه به‌روزرسانی‌های مداوم، به رشد و پایداری این ابزار کمک کرده‌اند.

از منظر مزایا، ReportPortal به دلیل متن‌باز بودن و قابلیت یکپارچه‌سازی آسان با چارچوب‌های تست رایج مانند JUnit، Pytest، TestNG و NUnit به‌سرعت در محیط‌های صنعتی پذیرفته شده است. داشبوردهای تعاملی و الگوریتم‌های هوش مصنوعی داخلی آن موجب کاهش چشمگیر زمان تحلیل و افزایش بهره‌وری تیم‌های QA و توسعه می‌شود. با این حال، معایبی نیز دارد؛ نصب و راه‌اندازی آن نیازمند منابع سخت‌افزاری قوی و دانش فنی در حوزه Docker و زیرساخت‌های ابری است و برای تیم‌های کوچک ممکن است پیچیده یا پرهزینه به نظر برسد. همچنین، هرچند قابلیت‌های هوش مصنوعی آن چشمگیر است، اما همچنان برای

^۱ Effective Programming for America

^۲ Root Cause Analysis

تحلیل‌های پیچیده به بازبینی انسانی نیاز وجود دارد. به‌طور کلی ReportPortal ابزاری قدرتمند برای سازمان‌های متوسط و بزرگ است که به دنبال تحلیل داده‌محور و هوشمند نتایج تست‌های خودکار هستند.

۳-۳-۴- ReportNG

یک افزونه سبک و متن‌باز برای چارچوب تست TestNG است که به‌طور خاص برای بهبود کیفیت گزارش‌های تولیدی این چارچوب توسعه یافته است. ایده‌ی اصلی ایجاد ReportNG از آنجا شکل گرفت که گزارش‌های پیش‌فرض TestNG ساده و فاقد امکانات بصری کافی بودند و تحلیلگران تست نیاز داشتند نتایج آزمون‌ها را با دقت و سهولت بیشتری بررسی کنند. بنابراین، جامعه توسعه‌دهندگان با هدف ارتقای قابلیت‌های گزارش‌دهی TestNG، ReportNG را به‌عنوان یک کتابخانه مکمل معرفی کرد. این ابزار با تولید گزارش‌های HTML و XML ساختاریافته، امکان مرور آسان‌تر تست‌های موفق و ناموفق را فراهم می‌سازد و در بسیاری از تیم‌های کوچک و متوسط که از TestNG استفاده می‌کنند، به‌عنوان یک راهکار عملیاتی محبوب شناخته می‌شود [۷].

در حال حاضر ReportNG در کنار سایر ابزارهای مکمل TestNG همچنان مورد استفاده قرار می‌گیرد، به‌ویژه در پروژه‌هایی که تمرکز اصلی بر تست‌های واحد^۱ و یکپارچه‌سازی^۲ با زبان Java دارند. اگرچه این ابزار امکانات پیشرفته‌ای مانند تحلیل خودکار خطا یا مصورسازی تعاملی نتایج را ارائه نمی‌دهد، اما به دلیل سادگی نصب و راه‌اندازی، گزارش‌های سبک و قابلیت ترکیب آسان با ابزارهای CI/CD همچون Jenkins، همچنان در محیط‌های توسعه حرفه‌ای جایگاه خود را حفظ کرده است [۷]. این ویژگی باعث می‌شود ReportNG بیشتر به‌عنوان یک مکمل ساده و قابل‌اعتماد برای تست‌های خودکار در نظر گرفته شود تا یک ابزار مستقل تحلیل نتایج.

مهم‌ترین نقطه قوت ReportNG سادگی، سرعت در تولید گزارش‌ها و سبک بودن آن است. این ابزار نیاز به پیکربندی پیچیده ندارد و در زمان کوتاهی می‌تواند خروجی‌های قابل‌فهمی را تولید کند که برای تیم‌های QA ارزشمند است. علاوه بر این، چون مبتنی بر TestNG طراحی شده، به‌خوبی با این چارچوب هماهنگ بوده و فرآیند ادغام آن در خط توسعه بسیار روان است. در مقابل، محدودیت‌های ReportNG نیز قابل توجه‌اند: نخست اینکه این ابزار تنها در بستر TestNG کاربرد دارد و برای چارچوب‌های تست دیگر مانند Junit یا Pytest قابل استفاده نیست. دوم اینکه فاقد قابلیت‌های پیشرفته تحلیل، مقایسه یا مصورسازی پویا است که ابزارهای مدرنی همچون Allure یا ReportPortal ارائه می‌دهند. به همین دلیل، ReportNG بیشتر مناسب

^۱ Unit Tests

^۲ Integration Tests

پروژه‌هایی است که تمرکز بر گزارش‌گیری ساده دارند و نیاز به تحلیل‌های عمیق و پیچیده احساس نمی‌شد.

۳-۵- Allure Report

این ابزار یک فریم‌ورک گزارش‌گیری متن‌باز است که توسط شرکت Yandex توسعه یافته و به‌عنوان یکی از محبوب‌ترین ابزارهای تولید گزارش تست در سطح جهانی شناخته می‌شود. ایده‌ی اصلی توسعه Allure از نیاز تیم‌های تست و توسعه به گزارش‌هایی تعاملی و قابل‌درک نشأت گرفت؛ چراکه گزارش‌های متنی سنتی غالباً پیچیده و برای مدیران پروژه یا تحلیلگران غیر فنی دشوار بودند. Allure با تولید گزارش‌های HTML تعاملی و کاربرپسند، نه تنها وضعیت تست‌ها را به‌طور دقیق نشان می‌دهد، بلکه با ارائه‌ی داشبوردهای تصویری، روند پیشرفت تست‌ها و نقاط بحرانی را به‌صورت گرافیکی به نمایش می‌گذارد [۸].

امروزه Allure در بسیاری از سازمان‌ها و پروژه‌های بزرگ نرم‌افزاری به‌کار گرفته می‌شود و با پشتیبانی از چارچوب‌های تست متنوع مانند JUnit، TestNG، Pytest، NUnit و حتی Cucumber، انعطاف‌پذیری بالایی در استفاده دارد. قابلیت یکپارچگی آسان با خطوط CI/CD مانند Jenkins، GitLab CI و TeamCity نیز یکی دیگر از دلایل محبوبیت گسترده آن است. علاوه بر این، Allure امکان نمایش تاریخچه اجرای تست‌ها، مقایسه چندین اجرا، و مستندسازی خودکار تست‌ها را فراهم می‌کند که این امر در پروژه‌های چابک^۱ و DevOps ارزش عملیاتی بسیار بالایی دارد.

Allure را می‌توان ابزاری جامع، سبک و قابل‌گسترش دانست. رابط کاربری جذاب، پشتیبانی گسترده از چارچوب‌ها و زبان‌های مختلف و قابلیت‌های بصری تعاملی از جمله مهم‌ترین نقاط قوت آن است. همچنین به‌دلیل متن‌باز بودن، کاربران می‌توانند به‌راحتی افزونه‌ها و سفارشی‌سازی‌های مورد نیاز خود را به آن اضافه کنند. اما در کنار این مزایا، محدودیت‌هایی نیز وجود دارد. به‌عنوان مثال، نصب و راه‌اندازی اولیه Allure ممکن است برای تیم‌های تازه‌کار چالش‌برانگیز باشد و نیاز به پیکربندی دقیق دارد. همچنین، این ابزار به‌تنهایی قادر به تحلیل عمیق خطاها نیست و بیشتر تمرکز آن بر نمایش و مدیریت داده‌های تست به‌شکل بصری است. در نتیجه، Allure ابزاری بسیار مناسب برای مصورسازی و مدیریت نتایج تست محسوب می‌شود، اما برای تحلیل هوشمند خطاها نیازمند ترکیب با سایر ابزارهای پیشرفته خواهد بود.

^۱ Agile

۳-۳-۶ - Allure TestOps

این محصول پیشرفته‌ای از اکوسیستم Allure است که توسط شرکت Qameta Software توسعه داده شد. این ابزار به‌عنوان یک پلتفرم مدیریت تست جامع طراحی شده است و برخلاف Allure Report که تمرکز آن صرفاً بر مصورسازی و گزارش‌دهی نتایج تست بود، TestOps با هدف پوشش‌دهی کل چرخه عمر تست شکل گرفت. ایده اصلی پشت توسعه این ابزار پاسخ به نیاز سازمان‌های بزرگ و تیم‌های DevOps برای داشتن یک بستر یکپارچه جهت مدیریت تست‌ها، تحلیل نتایج، و ارتباط مستقیم با مدیریت پروژه و فرآیند توسعه بود [۹]. به این ترتیب، TestOps ترکیبی از قابلیت‌های مدیریت تست سنتی و گزارش‌گیری مدرن Allure را در قالب یک پلتفرم واحد فراهم ساخت.

در وضعیت کنونی، Allure TestOps در بسیاری از تیم‌های بزرگ نرم‌افزاری و صنایع حساس همچون حوزه‌های مالی و فناوری به‌عنوان یک راهکار عملیاتی برای مدیریت تست مورد استفاده قرار می‌گیرد. این ابزار امکان طراحی، سازماندهی و اجرای تست‌ها را به‌طور مستقیم فراهم کرده و در کنار آن گزارش‌های تعاملی و مصورسازی مشابه Allure Report ارائه می‌دهد. یکی از نقاط قوت ویژه TestOps، پشتیبانی کامل از فرآیندهای Agile و DevOps است؛ به‌طوری که می‌تواند در خطوط CI/CD (مانند Jenkins، GitLab CI و Bamboo) ادغام شود و داده‌های تست را در لحظه پردازش نماید. همچنین، TestOps قابلیت یکپارچگی با ابزارهای مدیریت پروژه نظیر Jira را دارد که این امکان را می‌دهد تا تست‌ها مستقیماً به الزامات و نیازمندی‌های پروژه مرتبط شوند.

Allure TestOps ابزاری جامع و مدرن محسوب می‌شود که هم مدیریت و سازماندهی تست‌ها و هم مصورسازی نتایج را پوشش می‌دهد. رابط کاربری یکپارچه، انعطاف‌پذیری بالا، و پشتیبانی گسترده از چارچوب‌ها و زبان‌های مختلف، از جمله ویژگی‌های برجسته آن است. این ابزار همچنین امکان تحلیل روند تست‌ها، مقایسه اجرای متوالی و ایجاد داشبوردهای مدیریتی را برای تصمیم‌گیران فراهم می‌سازد. با این حال، محدودیت‌هایی نیز دارد. نخست اینکه برخلاف Allure Report، TestOps به‌طور کامل متن‌باز نیست و نسخه‌های پیشرفته آن نیازمند تهیه لایسنس تجاری است. دوم اینکه نصب و پیکربندی آن نسبت به Allure Report پیچیده‌تر است و برای بهره‌گیری کامل نیاز به دانش فنی بالاتری دارد. افزون بر این، به‌دلیل جامعیت بالای قابلیت‌ها، یادگیری و استقرار آن در سازمان‌ها ممکن است زمان‌بر باشد. در مجموع، Allure TestOps ابزاری قدرتمند برای تیم‌های حرفه‌ای QA و DevOps است که نیاز به یک بستر یکپارچه و مدیریتی دارند، اما هزینه و پیچیدگی آن می‌تواند مانعی برای تیم‌های کوچک‌تر یا تازه‌کار باشد.

۳-۴- تحلیل و مقایسه ابزارها

یکی از مهم‌ترین شاخص‌های مقایسه میان ابزارهای منتخب، سطح پوشش تست است. برخی ابزارها همچون TestComplete تمرکز اصلی خود را بر تست‌های رابط کاربری و رگرسیون قرار داده‌اند و از این منظر برای پروژه‌هایی که نیازمند بررسی تعاملی نرم‌افزار هستند بسیار کارآمدند. در مقابل، ابزارهایی مانند TestRail بیشتر نقش مدیریتی داشته و چرخه‌ی کلان تست را از مرحله طراحی تا تحلیل دربر می‌گیرند، هرچند برای اجرای مستقیم تست‌های واحد یا یکپارچه‌سازی نیازمند اتصال به ابزارهای جانبی هست. نیز با رویکردی داده‌محور، بیشترین تمرکز خود را بر تحلیل نتایج و ارائه دیدگاه‌های تحلیلی از شکست‌ها گذاشته است و از این طریق توانسته است به حوزه‌هایی مانند رگرسیون و تحلیل ریشه‌ای خطا کمک کند. ابزارهای سبک‌تر همچون ReportNG و Allure Report بیشتر در سطح گزارش‌دهی و ارائه‌ی نتایج تست‌های واحد و یکپارچه‌سازی به‌کار می‌روند و در صورت نیاز به امکانات مدیریتی پیشرفته‌تر باید در کنار ابزارهای دیگر مورد استفاده قرار گیرند. از منظر قابلیت گزارش‌دهی و تحلیل، تفاوت معناداری میان این ابزارها وجود دارد. TestComplete و Allure TestOps توانسته‌اند داشبوردهای تعاملی و گزارش‌های تصویری غنی ارائه دهند که برای تیم‌های توسعه و مدیریت بسیار ارزشمند است. ReportPortal با بهره‌گیری از الگوریتم‌های یادگیری ماشین، امکان دسته‌بندی خودکار خطاها و تحلیل ریشه‌ای را فراهم ساخته و همین موضوع آن را به یکی از ابزارهای مدرن در این حوزه تبدیل کرده است. در مقابل، ابزارهایی مانند ReportNG با وجود سادگی و سرعت در تولید خروجی، فاقد قابلیت‌های تحلیلی پیشرفته هستند و بیشتر به‌عنوان ابزاری مکمل برای چارچوب‌های تست مورد استفاده قرار می‌گیرند. بنابراین می‌توان گفت که انتخاب ابزار مناسب، به میزان نیاز سازمان به قابلیت‌های تحلیلی یا صرفاً گزارش‌دهی بستگی مستقیم دارد.

یکپارچگی با خطوط CI/CD معیار دیگری است که در سال‌های اخیر اهمیت ویژه‌ای یافته است. ابزارهایی همچون TestComplete، ReportPortal و Allure TestOps به‌صورت مستقیم با Jenkins، GitLab CI و Azure DevOps ادغام می‌شوند و از این طریق قادرند نتایج تست را در همان چرخه‌ی استقرار پردازش و تحلیل نمایند. در مقابل، ابزارهایی نظیر TestRail اگرچه با CI/CD یکپارچه می‌شوند، اما نقش اصلی آن‌ها بیشتر در سطح مدیریتی باقی می‌ماند و نیازمند اتصال به سایر ابزارهای اجرای تست هستند. ابزارهایی چون ReportNG یا Allure Report نیز با وجود قابلیت ادغام در خطوط CI/CD، بیشتر به‌عنوان بخشی از فرآیند گزارش‌گیری به کار می‌روند تا یک ابزار تحلیلی کامل. این مقایسه نشان می‌دهد که هر ابزار مزیت نسبی خاص خود را دارد و انتخاب آن باید متناسب با سطح بلوغ سازمان در استفاده از DevOps و نیاز به خودکارسازی تحلیل تست‌ها انجام گیرد. در جدول زیر خلاصه‌ای از مقایسه ابزارها بر اساس معیارهای یادشده ارائه شده است:

جدول ۲-۱: مقایسه ابزارها بر اساس معیارها

ابزار	سطح تست پوشش داده شده	قابلیت گزارش دهی و تحلیل	یکپارچگی با CI/CD
TestComplete	واحد، یکپارچه سازی، رابط کاربری، رگرسیون	گزارش های تصویری غنی، تحلیل تعاملی	یکپارچگی کامل با Jenkins و Azure
TestRail	مدیریت کل چرخه تست، بدون اجرای مستقیم	داشبورد مدیریتی، فاقد تحلیل ریشه ای	اتصال به CI/CD از طریق ابزارهای جانبی
ReportPortal	واحد، یکپارچه سازی، رگرسیون	تحلیل داده محور، دسته بندی خودکار خطا	ادغام مستقیم با Jenkins و GitLab CI
ReportNG	واحد و یکپارچه سازی در چارچوب (TestNG)	گزارش ساده و سریع، فاقد امکانات تحلیلی	امکان اتصال محدود در خطوط CI
Allure Report	واحد، یکپارچه سازی، رگرسیون	مصورسازی غنی، گزارش تعاملی	یکپارچگی آسان با ابزارهای CI/CD
Allure TestOps	واحد، یکپارچه سازی، مدیریت تست	داشبورد مدیریتی، تحلیل روند تست ها	ادغام کامل با Jenkins و Bamboo

۳-۵- ضرورت ایجاد ابزاری که در این پژوهش پیاده سازی شد

بررسی مقایسه ای ابزارهای موجود نشان می دهد که اگرچه هر یک از آنها قابلیت های ارزشمندی ارائه می دهند، اما هیچ کدام به طور کامل نیازهای تحلیل عمیق و هوشمند نتایج تست های مبتنی بر کد را پوشش نمی دهند. ابزارهایی مانند TestRail و Allure TestOps تمرکز بیشتری بر مدیریت کلان فرآیند تست دارند و ابزارهایی چون ReportNG یا Allure Report بیشتر نقش گزارش گر سبک را ایفا می کنند. حتی ابزارهای پیشرفته تر مانند ReportPortal نیز عمدتاً بر تحلیل داده های تست تمرکز دارند و کمتر به استخراج مسیرهای بحرانی کد یا تحلیل های مبتنی بر مدل های زبانی بزرگ پرداخته اند. همین خلأ، ضرورت توسعه ای یک سامانه ای یکپارچه را آشکار می سازد که بتواند علاوه بر گردآوری و نرمال سازی داده های تست، مسیرهای وابستگی کد را تحلیل کرده و با استفاده از LLM ها به شناسایی ریشه ای خطاها و ارائه ای پیشنهادهای اصلاحی بپردازد.

ابزار پیشنهادی در این پژوهش با تکیه بر همین ضرورت طراحی و پیاده سازی شده است. این سامانه با ترکیب قابلیت های گزارش دهی متداول و تحلیل داده محور پیشرفته، توانسته است خلأ موجود در ابزارهای پیشین را پر کند. رویکرد نوآورانه آن در استفاده از مدل های زبانی بزرگ برای تحلیل گزارش های تست، گامی فراتر از مصورسازی و دسته بندی خطاها برداشته و امکان تحلیل هوشمند و بافت محور نتایج را فراهم ساخته

است. به این ترتیب، پروژه حاضر نه تنها به عنوان یک ابزار تکمیلی، بلکه به عنوان پاسخی مستقیم به محدودیت‌های ابزارهای موجود قابل طرح است و می‌تواند نقطه شروعی برای ارتقای فرآیند تضمین کیفیت نرم‌افزار در پروژه‌های کد-محور باشد.

۳-۶- جمع‌بندی

در این فصل ابزارهای منتخب تست نرم‌افزار معرفی و ارزیابی شدند و مقایسه‌ها نشان داد که هیچ‌یک از آن‌ها توانایی پوشش جامع مدیریت، گزارش‌دهی و تحلیل ریشه‌ای را به‌طور هم‌زمان ندارند؛ ابزارهای مدیریتی صرفاً بر سازماندهی تمرکز دارند، ابزارهای گزارش‌محور به نمایش نتایج محدود می‌شوند و ابزارهای تحلیلی نیز عمق کافی در شناسایی مشکلات ندارند. این کاستی‌ها ضرورت توسعه راهکاری نوین را برجسته می‌سازد. پروژه حاضر با همین هدف و مبتنی بر مدل‌های زبانی بزرگ طراحی شده است تا به‌طور خاص برای برنامه‌نویسان و تسترها، امکان تحلیل دقیق و ریشه‌ای شکست‌ها را فراهم آورد و با ارائه دیدی روشن در فرآیند توسعه و تست، به ارتقای کیفیت نرم‌افزار کمک کند.

فصل چهارم

راهکار پیشنهادی

۴-۱- مقدمه

در این فصل روند پیاده‌سازی پروژه معرفی و تشریح می‌گردد. ابتدا نیازمندی‌های کاربردی و غیرکارکردی سامانه استخراج و بیان می‌شوند. سپس معماری کلی ابزار و اجزای اصلی آن شامل لایه خط فرمان، ماژول تحلیل و پردازش نتایج تست، ارتباط با مدل‌های زبانی بزرگ و بخش تولید گزارش توضیح داده خواهد شد. پس از آن مدل دامنه و موجودیت‌های اصلی معرفی می‌شوند و جریان کلی اجرای سامانه از دریافت لاگ‌های تست تا تولید گزارش نهایی تبیین خواهد شد. در ادامه جزئیات پیاده‌سازی هر بخش شامل پردازش ورودی‌ها، ساخت پرامپت ساختاریافته، فراخوانی LLM و رندر گزارش HTML/CSS/JS ارائه می‌گردد. در نهایت، نحوه اجرای سامانه از دید کاربر خط فرمان مرور شده و مسیر کلی اجرای ابزار به‌صورت یکپارچه جمع‌بندی می‌شود.

۴-۲- فهرست نیازمندی‌ها

۴-۲-۱- نیازمندی‌های کارکردی

R1) سیستم باید قادر به پذیرش ورودی‌های تست از فرمت‌های مختلف مانند Pytest JSON، JUnit XML و TRX/NUnit باشد.

R2) سیستم باید داده‌های ورودی تست‌ها را به صورت خودکار نرمال‌سازی کرده و به یک اسکیمای استاندارد {name, status, message} تبدیل کند.

R3) سیستم باید قادر به ساخت یک پرامپت ساختاریافته برای تحلیل با مدل‌های LLM (مانند AvalAI) باشد.

R4) سیستم باید قادر به تحلیل خودکار نتایج شکست‌ها با استفاده از مدل‌های LLM (مانند AvalAI) باشد.

- (R5) سیستم باید توانایی ارسال داده‌های ساختاریافته به مدل‌های LLM از جمله داده‌های مربوط به نوع خطا، ریشه‌یابی خطا، و پیشنهادات اصلاحی را داشته باشد.
- (R6) سیستم باید قادر به تولید گزارش HTML خودبسنده و تعاملی باشد که شامل اطلاعات KPI مانند تعداد Passed، Failed، Skipped و جداول و نمودارهای خطاها باشد.
- (R7) سیستم باید قادر به تحلیل و تشخیص مسیرهای بحرانی^۱ بر اساس داده‌های تست شکست‌خورده باشد.
- (R8) سیستم باید قادر به دریافت و پردازش ورودی‌ها از طریق CLI با پارامترهای --project, --log, --lang, --model, --open-report باشد.
- (R9) سیستم باید تحلیل‌های LLM را بر اساس ورودی‌های ساختاریافته مانند نام تست، نوع خطا و مسیر بحرانی تولید کند.
- (R10) سیستم باید قادر باشد هر نوع مدل LLM را با API Key شخصی هر شخص، برای بهینه‌سازی نتایج تحلیل تنظیم کند.
- (R11) سیستم باید اطلاعات محرمانه مانند API Key را از گزارش نهایی و فایل‌های ذخیره شده حذف کند تا از نظر امنیتی محافظت شود.

۴-۲-۲- نیازمندی‌های غیرکارکردی

- (R1) سیستم باید قابلیت عملکرد سریع برای پردازش لاگ‌های بزرگ (تعداد بالا از تست‌ها) را داشته باشد.
- (R2) سیستم باید از قابلیت توسعه‌پذیری بالایی برخوردار باشد تا در آینده بتوان زبان‌های جدید یا فرمت‌های جدید لاگ را به آن اضافه کرد.
- (R3) سیستم باید قابلیت اجرا و عملکرد مناسب در پلتفرم‌های مختلف عملیاتی مانند Linux، Windows و macOS را داشته باشد.
- (R4) سیستم باید قادر باشد به‌طور خودکار در محیط‌های CI/CD اجرا شود و گزارش‌هایی از این محیط‌ها تولید کند.
- (R5) سیستم باید قابلیت تولید گزارش‌هایی با طراحی جذاب و گرافیکی را داشته باشد تا به‌راحتی برای تیم‌های QA و توسعه‌دهندگان قابل درک باشد.
- (R6) سیستم باید دارای مستندات کامل برای نصب، راه‌اندازی و استفاده باشد، به‌طوری که کاربران جدید بتوانند به راحتی سیستم را نصب و استفاده کنند.
- (R7) سیستم باید قادر به پردازش ورودی‌ها و اجرای فرآیندها به صورت همزمان^۲ باشد تا زمان پردازش به

^۱ Critical Paths

^۲ parallel processing

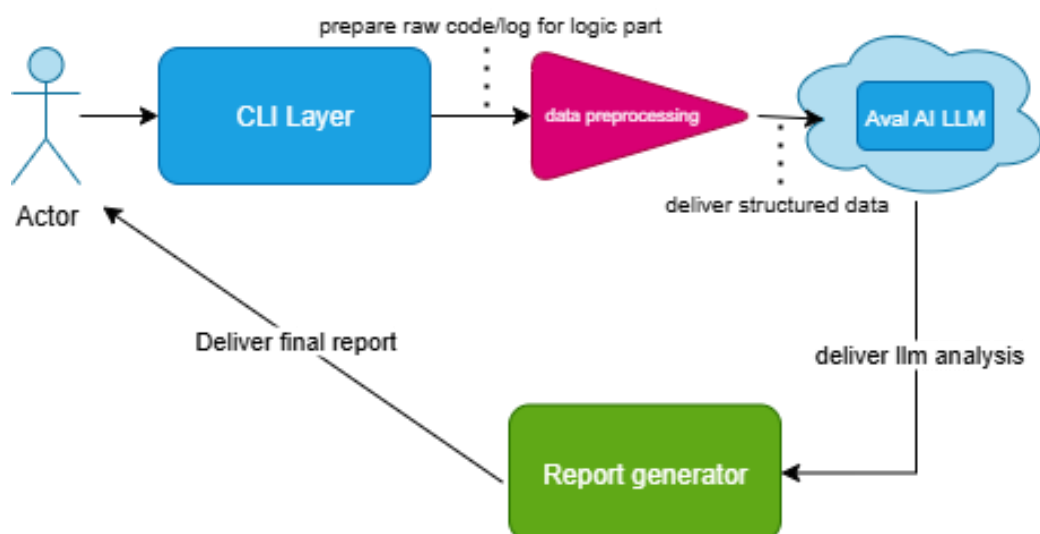
حداقل برسد.

۴-۳- معماری سیستم

در معماری این پروژه، تمامی اجزای سیستم به‌طور یکپارچه و در قالب یک پروژه واحد توسعه و پیاده‌سازی شده‌اند. این سیستم به‌طور کلی شامل لایه‌های مختلفی است که هرکدام وظیفه خاص خود را در پردازش داده‌ها و تحلیل‌های حاصل از آن‌ها بر عهده دارند. تمام اجزا به‌طور مستقل از یکدیگر عمل کرده، اما از طریق یکپارچگی در سطح سیستم به‌طور هماهنگ به نتیجه نهایی می‌رسند.

معماری سیستم این پروژه همانطور که در شکل ۴-۱ نشان داده شده است، به‌گونه‌ای طراحی شده است که تمامی فرآیندها به‌صورت مرحله‌ای و بدون نیاز به ذخیره‌سازی دائمی داده‌ها انجام می‌شود. ورودی‌ها از طریق لایه رابط کاربری خط فرمان دریافت شده و به ماژول‌های مختلف سیستم ارسال می‌شوند. داده‌ها پس از نرمال‌سازی توسط ماژول پردازش لاگ‌ها، به فرمت استاندارد تبدیل شده و برای ساخت پرامپت ساختاریافته به ماژول ساخت پرامپت ارسال می‌شوند. این پرامپت‌ها سپس به مدل‌های LLM برای تحلیل ارسال می‌شوند که نتایج آن شامل ریشه‌یابی خطاها، شناسایی مسیرهای بحرانی و پیشنهادات اصلاحی است.

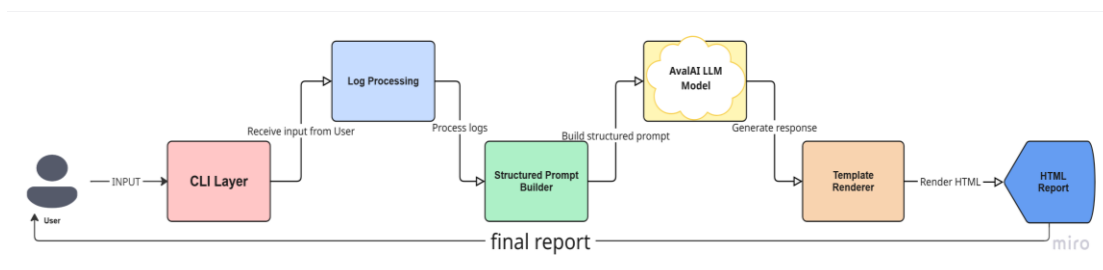
پس از دریافت نتایج تحلیل، ماژول تولید گزارش HTML گزارشی نهایی شامل تحلیل‌ها، نمودارها، جداول و اطلاعات KPI تولید می‌کند. این فرآیند به‌صورت سلسله‌مراتبی انجام شده و در نهایت کاربر گزارشی تعاملی و خودبسنده مشاهده می‌کند. معماری این سیستم به‌گونه‌ای طراحی شده است که قابلیت مقیاس‌پذیری بالایی دارد و به راحتی می‌توان آن را به پروژه‌ها و سیستم‌های دیگر گسترش داد. تمامی اجزای سیستم بدون نیاز به ذخیره‌سازی پایدار داده‌ها و پیچیدگی‌های ارتباطات بین فرآیندی به یکدیگر متصل هستند و این انعطاف‌پذیری باعث ساده‌تر شدن نگهداری و گسترش سیستم می‌شود.



شکل ۴-۱: معماری سامانه تحلیل نتایج آزمون

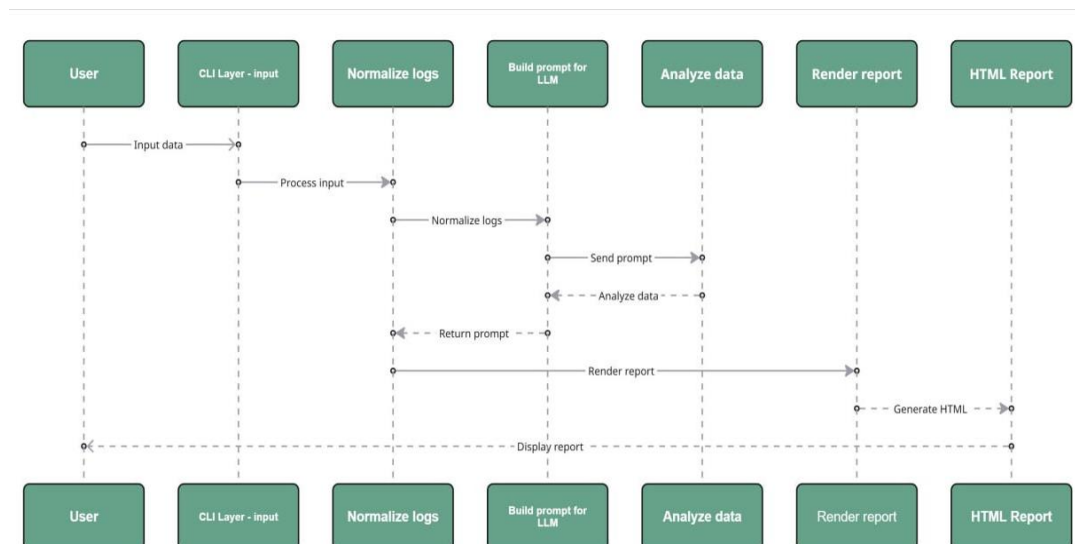
۴-۴- روند کلی سامانه

پس از بررسی دقیق نیازمندی‌ها و طراحی ساختار سیستم، مطابق شکل ۴-۲، فرآیند کلی اجرای سامانه به صورت گام به گام تعریف شد. ابتدا کاربر وارد سیستم از طریق CLI می‌شود، جایی که قادر است دستورات مورد نظر خود را برای پردازش لاگ‌ها و تحلیل داده‌ها وارد کند. در این مرحله، ورودی‌هایی مانند مسیر پروژه، فایل‌های لاگ تست و پارامترهای مدل LLM از کاربر دریافت می‌شود و وارد سامانه می‌شود. سپس داده‌ها از طریق ماژول‌های مختلف پردازش می‌شوند. داده‌های ورودی ابتدا به ماژول پردازش لاگ‌ها ارسال می‌شوند تا به فرمت استاندارد تبدیل شوند. پس از نرمال‌سازی این داده‌ها، سیستم مرحله بعدی را آغاز می‌کند که شامل ساخت پرامپت ساختاریافته است. در این مرحله، یک پرامپت ساختاریافته که حاوی اطلاعات ضروری مانند نام تست‌ها، نتایج آن‌ها و پیام‌های خطا است، ایجاد شده و با api key به یکی از مدل‌های AvalAI ارسال می‌شود. مدل AvalAI پس از دریافت پرامپت‌ها، تحلیل‌های مورد نظر خود را انجام داده و نتایج تحلیل را به سامانه باز می‌گرداند. این تحلیل‌ها شامل ریشه‌یابی خطاها، شدت آن‌ها و پیشنهادات اصلاحی هستند. این نتایج سپس به Report Renderer ارسال می‌شوند. در این مرحله، ماژول Report Renderer مسئول رندر کردن داده‌ها به یک گزارش نهایی HTML است که شامل اطلاعات KPI، جداول و نمودارهای مرتبط با تست‌ها است. گزارشی نهایی به صورت خودبسنده^۱ و تعاملی به کاربر ارائه می‌شود تا او بتواند تحلیل‌ها را مشاهده و بررسی کند. تمام این فرآیند به صورت یکپارچه و بدون نیاز به ذخیره‌سازی دائمی داده‌ها انجام می‌شود و تنها خروجی‌های پردازش‌شده در قالب فایل‌های HTML نهایی ذخیره می‌شوند. مطابق با شکل ۴-۳ این طراحی باعث می‌شود که این فرآیندها به طور متوالی و کارآمد انجام شوند و در نهایت کاربر از نتایج تحلیل‌ها بهره‌برداری کند.



شکل ۴-۲: فرآیند کلی پردازش و تحلیل داده‌های آزمون در سامانه

^۱ Self-contained Reports



شکل ۴-۳: نمودار توالی اجرای فرآیند تحلیل خطا در سامانه Failtrace

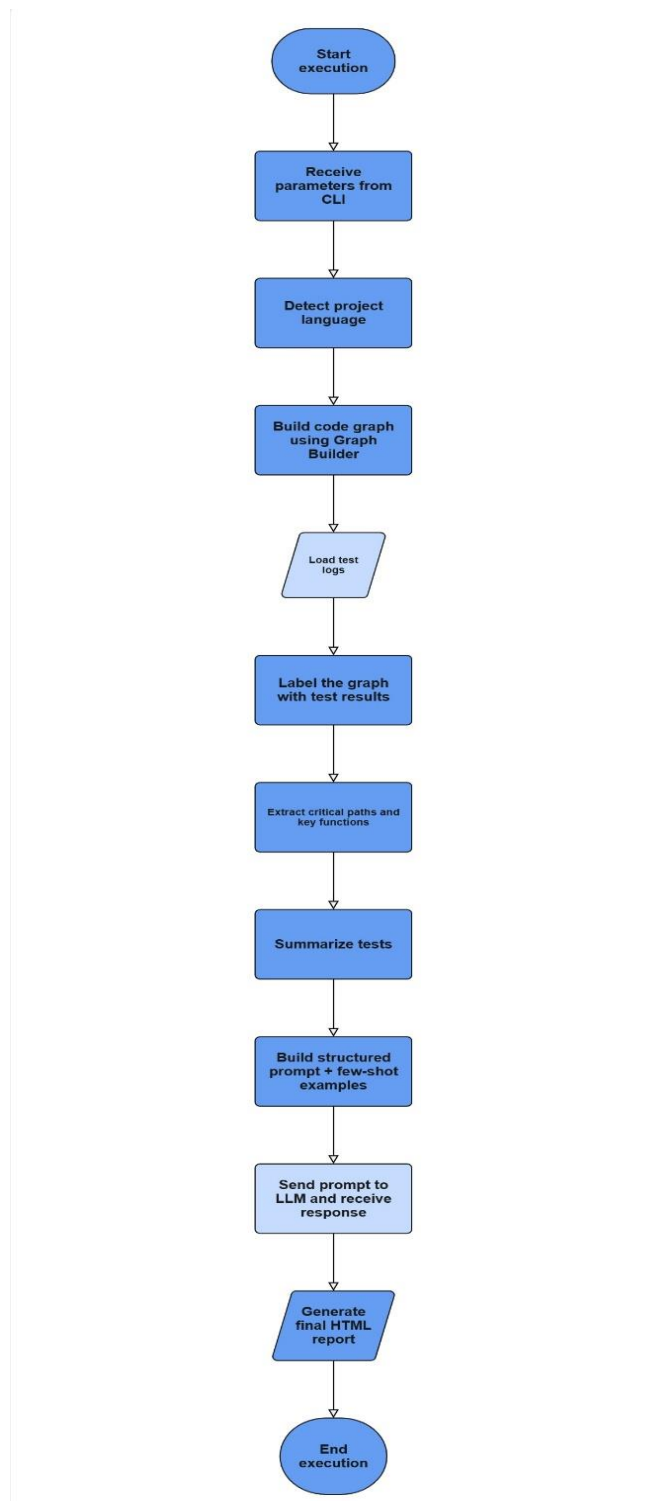
۴-۵- پیاده‌سازی سیستم

در این بخش، مراحل پیاده‌سازی سیستم به تفصیل شرح داده می‌شود. این فرآیند از طراحی معماری سیستم تا پیاده‌سازی دقیق الگوریتم‌ها، پردازش داده‌ها و تولید گزارش‌های نهایی را شامل می‌شود. سیستم طراحی شده به گونه‌ای است که تمامی داده‌ها به‌طور خودکار از منابع ورودی (فایل‌های لاگ تست) پردازش شده، نرمال‌سازی می‌شوند و به‌طور ساختاریافته به مدل‌های LLM ارسال می‌شوند. در نهایت، خروجی‌های تحلیل‌شده به گزارش‌هایی تعاملی و خودبسنده در قالب HTML تبدیل می‌شوند. این پیاده‌سازی به‌صورت یکپارچه و بدون نیاز به ذخیره‌سازی دائمی داده‌ها انجام می‌شود و تمامی فرآیندها در یک چرخه مشخص و کارآمد پیاده‌سازی شده‌اند.

۴-۵-۱- پیاده‌سازی ماژول CLI

ماژول رابط خط فرمان یا همان CLI نقطه‌ی آغازین کاربری سامانه محسوب می‌شود. این ماژول که در فایل main.py پیاده‌سازی شده و از طریق فایل اجرایی main.py نیز قابل فراخوانی است، امکان برقراری ارتباط مستقیم کاربر با سیستم را فراهم می‌سازد. طراحی CLI به گونه‌ای صورت گرفته است که بتواند هم فرآیند

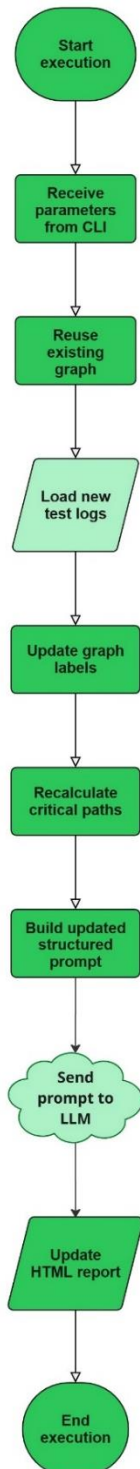
کامل تحلیل و گزارش گیری را به انجام برساند و هم در شرایطی که کاربر صرفاً به یک اجرای سریع و به روزرسانی گزارش نیاز دارد، پاسخگو باشد. از این رو دو حالت اصلی در آن تعریف گردیده است: حالت کامل و حالت سریع. در حالت کامل مطابق شکل ۴-۴، تمامی مراحل چرخه‌ی کاری سامانه از نقطه‌ی شروع تا پایان اجرا می‌گردد. در این حالت ابتدا زبان پروژه تشخیص داده می‌شود و در ادامه گراف وابستگی کد به کمک سازنده‌های زبان محور تولید می‌گردد. پس از آن لاگ‌های تست بارگذاری شده و نتایج آن‌ها روی گراف برچسب‌گذاری می‌شود. مسیرهای بحرانی و توابع کلیدی استخراج می‌شوند، خلاصه‌سازی تست‌ها انجام می‌گیرد و پرامپت ساختاریافته به همراه نمونه‌های آموزشی به مدل زبانی بزرگ ارسال می‌شود. خروجی نهایی این فرآیند یک گزارش کامل و غنی است که تمامی بخش‌های لازم برای تحلیل ریشه‌ای خطاها را در بر دارد. حالت کامل بیشتر برای دفعات اولیه‌ی اجرا روی یک پروژه یا زمانی که تغییرات اساسی در کد منبع رخ داده باشد مناسب است.



شکل ۴-۴: اجرای ابزار Failtrace در حالت کامل از طریق خط فرمان

در مقابل مطابق شکل ۴-۵ حالت سریع طراحی شده است تا زمان پردازش کاهش یابد و گزارش به شکل کارآمدتری به روزرسانی شود. در این حالت، گراف تولیدشده در اجرای قبلی مجدداً مورد استفاده قرار می‌گیرد

و صرفاً لاگ‌های جدید تست بر روی آن اعمال می‌شوند. سپس مسیرهای بحرانی بازمحاسبه شده و گزارش نهایی به‌روزرسانی می‌گردد. این قابلیت باعث می‌شود که در فرآیندهای توسعه‌ی مستمر و یکپارچه‌سازی مداوم، سامانه بتواند در کوتاه‌ترین زمان ممکن بازخوردهای جدید را در اختیار تیم توسعه قرار دهد.



شکل ۴-۵: اجرای ابزار Failtrace در حالت سریع از طریق خط فرمان

از منظر فنی، CLI مجموعه‌ای از پارامترهای اختیاری و اجباری را دریافت می‌کند که امکان شخصی‌سازی اجرای سامانه را برای کاربر فراهم می‌آورند. مسیر پروژه، مسیر لاگ‌ها، زبان برنامه‌نویسی، مدل زبانی انتخابی و گزینه‌هایی همچون اجرای آزمایشی با آپشن Dry-run و باز کردن خودکار گزارش خروجی از جمله این پارامترها هستند. طراحی رابط خط فرمان به گونه‌ای بوده است که حتی کاربران کم‌تجربه نیز بتوانند با چند دستور ساده از تمامی قابلیت‌های سامانه بهره‌مند شوند. در عین حال، پیام‌های راهنمای CLI در صورت بروز خطا یا استفاده‌ی نادرست از پارامترها به کاربر نمایش داده می‌شود و از این طریق تجربه‌ی کاربری بهتری فراهم می‌گردد.

```
PS C:\Users\Mohadese\Desktop\project> $env:AVALAI_API_KEY="aa-wmyb0LAiQcHi1GGT4kMjCjhY9xl4kXAVeC4bdPitj0Mep68K"
PS C:\Users\Mohadese\Desktop\project> python -m failtrace full --open-report -p C:\Users\Mohadese\RiderProjects\Tik-SE\Service -l C:\Users\Mohadese\RiderProjects\Tik-SE\Service\Application.Tests\TestResults\TestResults.xml
```

شکل ۴-۶: نمونه‌ای از اجرای ابزار Failtrace در حالت کامل از طریق خط فرمان

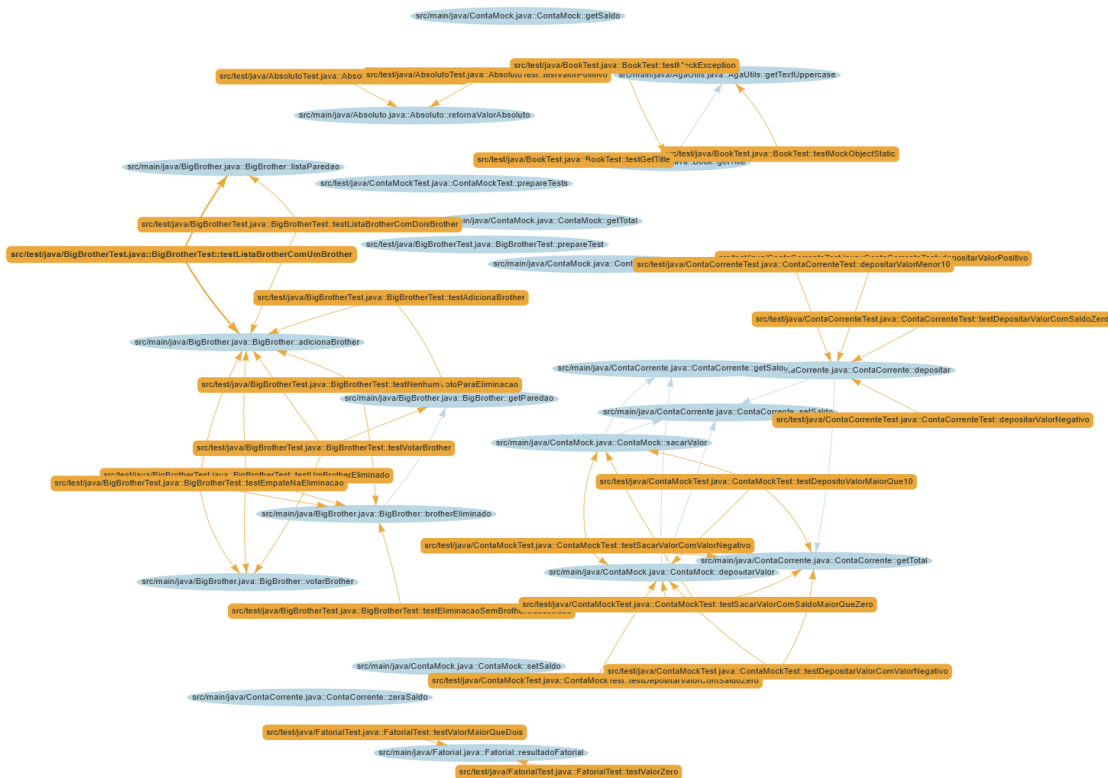
این ساختار منسجم و انعطاف‌پذیر موجب شده است که ماژول CLI به‌عنوان یکی از اجزای کلیدی سامانه نه تنها نقش نقطه‌ی ورود، بلکه نقش تنظیم‌کننده و هماهنگ‌کننده‌ی کل فرآیند را ایفا کند. به بیان دیگر، CLI همانند یک کنترل‌کننده مرکزی عمل کرده و مسیر حرکت داده‌ها و پردازش‌ها را از مرحله‌ی ورودی تا تولید گزارش نهایی هدایت می‌نماید.

۴-۵-۲- پیاده‌سازی ماژول تحلیل و گراف کد

یکی از بخش‌های کلیدی در طراحی سامانه Failtrace، ماژول تحلیل و گراف کد است. هدف اصلی این ماژول آن است که کد منبع پروژه را از حالت متنی خطی به یک نمایش ساختاریافته در قالب گراف تبدیل کند تا بتوان وابستگی‌ها، مسیرهای اجرایی و ارتباط میان اجزای برنامه را به شکلی دقیق و قابل پردازش مدل‌سازی کرد. اهمیت این ماژول در آن است که بسیاری از تحلیل‌های بعدی سامانه، مانند استخراج مسیرهای بحرانی یا ارتباط تست‌های شکست‌خورده با توابع معیوب، وابسته به درستی و دقت این لایه‌ی گرافی است. در واقع می‌توان گفت این ماژول ستون فقرات سامانه محسوب می‌شود و بدون آن امکان ایجاد بینش عمیق نسبت به خطاهای نرم‌افزار وجود نخواهد داشت.

فرآیند کار در این بخش به این صورت است که ابتدا کد منبع به‌عنوان ورودی دریافت می‌شود. برای هر زبان برنامه‌نویسی، یک ماژول اختصاصی پیاده‌سازی شده است تا بتواند قواعد نحوی و ساختاری همان زبان را درک کند. به همین دلیل ماژول‌هایی نظیر `python_graph_builder`، `java_graph_builder` و `csharp_graph_builder` طراحی گردیده‌اند که هر یک وظیفه‌ی تحلیل و گراف‌سازی کد به زبان متناظر را بر

عهده دارند. این ماژول‌ها با استفاده از تحلیل ایستا^۱ ساختار کد را بررسی کرده و نودهایی را برای توابع، کلاس‌ها و ماژول‌ها تولید می‌کنند. سپس لبه‌ها^۲ به گونه‌ای ترسیم می‌شوند که وابستگی‌ها، فراخوانی‌ها و توالی اجرای کد به‌طور کامل مشخص گردد.



شکل ۴-۷: نمونه‌ای از گراف وابستگی تولیدشده برای یک پروژه نرم‌افزاری

به‌منظور استانداردسازی و جلوگیری از پراکندگی، تمامی این ماژول‌های زبان‌محور بر پایه یک هسته مشترک توسعه یافته‌اند که در فایل `base.py` تعریف شده است. این لایه پایه باعث می‌شود که گراف‌های تولیدی صرف‌نظر از زبان برنامه‌نویسی، ساختاری یکنواخت داشته باشند و در مراحل بعدی بتوان به راحتی آن‌ها را پردازش و مقایسه نمود. افزون بر این، برای تکمیل فرآیند تحلیل، ماژول‌هایی نظیر `detector.py` و `plugins.py` نیز در نظر گرفته شده‌اند تا قابلیت شناسایی الگوهای خاص و افزونه‌های تحلیلی در سطح گراف فراهم گردد (شکل ۴-۸ و شکل ۴-۹).

¹ Static Analysis

² Edge

```
def detect_language(project_path: str) -> Literal["python", "java", "csharp", "unknown"]:
    extensions = set()
    for root, _, files in os.walk(project_path):
        for file in files:
            _, ext = os.path.splitext(file.lower())
            extensions.add(ext)

    if ".py" in extensions:
        return "python"
    elif ".java" in extensions:
        return "java"
    elif ".cs" in extensions:
        return "csharp"
    else:
        return "unknown"
```

شکل ۴-۸: کد detector.py

```
def register(lang: str):
    def decorator(cls: Type[GraphBuilder]):
        _registry[lang.lower()] = cls
        return cls

    return decorator

def get_builder(lang: str) -> GraphBuilder:
    try:
        builder_cls = _registry[lang.lower()]
    except KeyError:
        raise ValueError(f"No GraphBuilder registered for language '{lang}'")
    return builder_cls()

from .python_graph_builder import PythonGraphBuilder
from .java_graph_builder import JavaGraphBuilder
from .csharp_graph_builder import CSharpGraphBuilder
```

شکل ۴-۹: کد plugin.py

یکی از ویژگی‌های برجسته در این بخش، امکان تلفیق نتایج تحلیل کد با داده‌های مربوط به تست‌ها است. زمانی که لاگ‌های اجرای تست بارگذاری می‌شوند، بر روی گراف تولید شده برچسب‌گذاری صورت می‌گیرد تا مشخص شود کدام گره‌ها یا مسیرها با خطا مواجه شده‌اند. این تلفیق سبب می‌شود که گراف نه‌تنها بیانگر ساختار ایستا باشد، بلکه اطلاعات پویای اجرای واقعی برنامه را نیز در خود جای دهد. به این ترتیب، تحلیل خطا از یک بررسی سطحی فراتر رفته و به یک تحلیل عمیق و مبتنی بر شواهد اجرایی تبدیل می‌شود. در مجموع می‌توان بیان کرد که ماژول تحلیل و گراف کد، نقطه‌ی تلاقی دو رویکرد ایستا و پویا در تحلیل نرم‌افزار است. این ماژول توانسته است با ایجاد یک نمایش تصویری و قابل‌کاوش از کد و خطاها، زمینه‌ساز بخش‌های بعدی مانند استخراج مسیر بحرانی، تولید پرامپت‌های ساختاریافته و در نهایت گزارش‌دهی دقیق

شود. انتخاب معماری ماژولار و طراحی جداگانه برای هر زبان برنامه‌نویسی از یک سو، و یکپارچه‌سازی آن‌ها در قالب گرافی استاندارد از سوی دیگر، این امکان را فراهم کرده است که سامانه Failtrace از انعطاف‌پذیری بالا و دقت تحلیلی قابل توجهی برخوردار باشد.

۴-۵-۳- پردازش لاگ‌های تست و نرمال‌سازی

یکی از مراحل مهم در چرخه‌ی اجرای ابزار Failtrace، پردازش لاگ‌های تست و تبدیل آن‌ها به داده‌ای ساختارمند و قابل تحلیل است. خروجی تست‌ها بسته به زبان برنامه‌نویسی و فریم‌ورک استفاده‌شده می‌تواند شامل اطلاعات متنوعی مانند شناسه‌ی تست، وضعیت اجرا، جزئیات خطاها و زمان‌بندی باشد. این داده‌های خام عموماً پراکنده، ناهمگون و دشوار برای پردازش مستقیم هستند و همین امر ضرورت وجود مرحله‌ای برای یکپارچه‌سازی و نرمال‌سازی آن‌ها را آشکار می‌سازد. هدف از این مرحله، آماده‌سازی داده‌هایی استاندارد است که بتوانند به‌طور یکنواخت در مراحل بعدی تحلیل و ترسیم گراف کد به کار گرفته شوند.

```
{
  "total_tests": 44,
  "executed_tests": 21,
  "passed_tests": 20,
  "failed_tests": 1,
  "skipped_tests": 0,
  "untagged_tests": 23,
  "failed_detail": [
    {
      "name": "Application.Tests.ServiceTests::Application.Tests/ServiceTests/EventServiceTests.cs::EventServiceTests::Ge
      "file": "Application.Tests/ServiceTests/EventServiceTests.cs",
      "line": null,
      "error": "Expectation has property AnyoneCanAddSelf that the other object does not have.\nExpectation has property
      "doc": ""
    }
  ]
}
```

شکل ۴-۱۰: ساختار فایل summary.json شامل داده‌های لاگ آزمون

برای تحقق این هدف، دو ماژول logs_parser.py و normalize.py طراحی و پیاده‌سازی شده‌اند. ماژول logs_parser مسئولیت استخراج و بازخوانی گزارش‌های تست در قالب‌های متداولی همچون XML و JSON را بر عهده دارد و آن‌ها را به یک ساختار میانی استاندارد تبدیل می‌کند. پس از آن، ماژول normalize داده‌های استخراج‌شده را پالایش کرده، مقادیر اضافی و ناسازگار را حذف و در نهایت مجموعه‌ای یکپارچه و منسجم تولید می‌نماید. خروجی این مرحله داده‌هایی است که ضمن همگن بودن، امکان اتصال مستقیم به گراف کد و برچسب‌گذاری بخش‌های مختلف آن را فراهم می‌سازد و بدین ترتیب پایه‌ای مطمئن برای تحلیل‌های بعدی به وجود می‌آید.

نتیجه این فرآیند، کاهش خطاهای تحلیلی و افزایش دقت در شناسایی بخش‌های معیوب کد است. نرمال‌سازی مانع از تناقض در داده‌ها می‌شود، به‌ویژه در مواردی که تست‌ها با شناسه‌های متفاوت اما عملکرد یکسان گزارش می‌شوند یا زمان‌های ثبت‌شده موجب تداخل تفسیری می‌گردد. با انتقال داده‌های نرمال‌شده به گراف و برچسب‌گذاری گره‌ها، ارتباط میان تست‌ها و اجزای کد به‌روشنی آشکار می‌شود. به این ترتیب، تحلیل خطا دیگر محدود به متون خام گزارش نیست، بلکه در بستر گراف کد به‌صورت ساختاریافته قابل مشاهده خواهد بود؛ عاملی که دقت و عمق تحلیل Failtrace را به‌طور چشمگیری افزایش می‌دهد.

۴-۵-۴- استخراج مسیرهای بحرانی و توابع کلیدی

یکی از مراحل مهم در سامانه Failtrace، شناسایی مسیرهای بحرانی و توابع کلیدی است که به‌ویژه در تحلیل‌های عمیق از شکست‌های تست و اشکالات کد استفاده می‌شود. این مرحله بر اساس گراف کدی که در مراحل قبلی ساخته شده، عمل می‌کند و با هدف شناسایی نواحی کد که تأثیر بیشتری بر روی نتایج تست‌ها دارند، طراحی شده است. مسیرهای بحرانی به مسیری گفته می‌شود که از آن‌ها به‌طور مداوم تست‌های شکست‌خورده عبور می‌کنند، در حالی که توابع کلیدی توابعی هستند که در این مسیرها بیشترین فراخوانی و نقش را دارند. شناسایی این توابع و مسیرها به تیم توسعه کمک می‌کند تا دقیقاً بدانند کدام قسمت‌های کد نیاز به بازبینی یا بهبود دارند.

در سامانه Failtrace، این فرآیند به کمک ماژول `critical_path_extractor.py` پیاده‌سازی شده است. این ماژول با استفاده از گراف‌های تولیدشده در مراحل قبلی، مسیرهایی را که بیشترین وابستگی به تست‌های شکست‌خورده دارند، شناسایی می‌کند. به‌منظور محاسبه این مسیرها، ابتدا باید از الگوریتم‌های گرافی مانند `networkx.all_simple_paths` استفاده کرد که به‌طور خودکار تمامی مسیرهای ممکن در گراف را شبیه‌سازی می‌کند و سپس بر اساس شکست‌های تست، مسیرهایی که بیشترین ارتباط با آن‌ها دارند، استخراج می‌شوند. این مسیرها نه‌تنها وابستگی‌های مستقیم را نشان می‌دهند بلکه پیوندهای غیرمستقیم و تأثیرات جانبی در کد را نیز مشخص می‌کنند.

نکته‌ای که در استخراج مسیرهای بحرانی حائز اهمیت است، شناسایی نودهای `external` است که در گراف به‌عنوان نقاط خارجی شناخته می‌شوند. این نودها معمولاً به‌صورت بخش‌هایی از کد یا کتابخانه‌های خارجی نمایش داده می‌شوند که نمی‌توانند به‌طور مستقیم تغییر داده شوند. برای تجزیه و تحلیل این نودها و تعیین اهمیت آن‌ها در مسیر بحرانی، از ماژول `path_enricher.py` استفاده می‌شود. این ماژول با افزودن فراداده^۱ به مسیرهای بحرانی و توابع کلیدی، اطلاعات بیشتری در اختیار تحلیلگران قرار می‌دهد. به‌عنوان مثال، در صورتی

^۱ Metadata

که یک تابع خارجی نقش کلیدی در مسیر بحرانی ایفا کند، این اطلاعات در مسیر به‌طور واضح‌تری نمایان می‌شود تا تحلیلگران بتوانند به‌طور دقیق‌تر مشکلات را شبیه‌سازی کنند.

برای شناسایی توابع کلیدی، ماژول `function_extractor.py` به‌کار می‌رود که به‌طور خاص توابع موجود در مسیرهای بحرانی را استخراج می‌کند. این ماژول با شبیه‌سازی رفتار کد و فراخوانی‌های توابع، توابعی که بیشتر از بقیه در مسیرهای بحرانی تکرار می‌شوند را شناسایی می‌کند. این توابع می‌توانند نمایانگر نقاط ضعف سیستم باشند و به تیم توسعه نشان دهند که کدام قسمت‌های کد باید بهینه‌سازی یا بازنگری شوند. در نتیجه، این فرآیند موجب می‌شود که کد به‌طور مؤثرتری مورد بررسی قرار گیرد و تحلیل‌های بهتری در مورد محل دقیق اشکالات نرم‌افزاری انجام شود.

```

output > function_summaries.json > ...
{
  "infrastructure/Repositories/UserScheduleRepository.cs::UserScheduleRepository::GetAsync": {
    "file": "infrastructure/Repositories/UserScheduleRepository.cs",
    "line": 48,
    "docstring": "",
    "code": "    public async Task<List<UserSchedule>> GetAsync(long userId, DayOfWeek? day = null) {        var query = _context.UserSchedules
  },
  "infrastructure/Integrations/GoogleEventProvider.cs::GoogleEventProvider::GetAsync": {
    "file": "infrastructure/Integrations/GoogleEventProvider.cs",
    "line": 16,
    "docstring": "",
    "code": "    public async Task<Event> GetAsync(string calendarId, string eventId) {        var status = (await _calendarService.Events.Get(calendarId, eventId)
  },
  "Application/Services/EventService.cs::EventService::GetAsync": {
    "file": "Application/Services/EventService.cs",
    "line": 21,
    "docstring": "",
    "code": "    public async Task<OperationResult<EventResponse>> GetAsync(string calendarId, string eventId) {        var anEvent = await _googleEventProvider.GetAsync(calendarId, eventId)
  },
  "Domain/Exceptions/EventNotFoundException.cs::EventNotFoundException::EventNotFoundException": {
    "file": "Domain/Exceptions/EventNotFoundException.cs",
    "line": 9,
    "docstring": "",
    "code": "    public EventNotFoundException() : base(\"Event Not Found Exception\",new Guid(\"44EECF82-C17F-44DE-9410-1062B36C1BAC\")) {
  }
}

```

شکل ۴-۱۱: خروجی استخراج توابع کلیدی در مسیرهای بحرانی یک پروژه نرم‌افزاری

با تکمیل این فرآیند، سامانه `Failtrace` می‌تواند گزارشی دقیق و مستند از مسیرهای بحرانی و توابع کلیدی تولید کند که در آن علاوه بر مسیرهای بحرانی، توابع مرتبط، میزان تأثیرگذاری هر تابع در شکست تست‌ها، و نودهای خارجی نیز به‌طور کامل مشخص شده‌اند. این گزارش‌ها به‌ویژه برای پروژه‌های بزرگ که شامل صدها یا هزاران خط کد هستند، بسیار مفید هستند چرا که تحلیلگر را قادر می‌سازند به‌طور مؤثری تنها بخش‌های بحرانی کد را مورد بررسی قرار دهد.

۴-۵-۵- ساخت پرامیت ساختاریافته و ارتباط با LLM

ساخت پرامیت ساختاریافته و برقراری ارتباط مؤثر با مدل‌های زبانی بزرگ یکی از ارکان اساسی در تحلیل دقیق و هوشمندانه خطاها و شکست‌های تست در سامانه Failtrace است. پس از آن که گراف وابستگی کد و اطلاعات مربوط به نتایج تست پردازش شدند، گام بعدی در فرآیند تحلیل، تبدیل این داده‌ها به یک ساختار مناسب برای ارسال به LLM است. هدف اصلی از ساخت این پرامیت‌ها، ارائه اطلاعات به‌صورت قابل فهم برای مدل‌های زبانی است تا این مدل‌ها بتوانند تحلیل‌های عمیق‌تری انجام داده و پاسخ‌های مفیدی ارائه دهند. برای این منظور، ماژول structured_prompt_builder.py طراحی شده است. مطابق شکل‌های ۴-۱۲ و ۴-۱۳ این ماژول به‌طور خودکار داده‌های مختلف مربوط به پروژه، از جمله آمار تست‌ها، مسیرهای بحرانی و توابع کلیدی، را جمع‌آوری کرده و یک پرامیت ساختاریافته برای LLM می‌سازد. این پرامیت به‌طور دقیق طراحی شده است تا تمامی اطلاعات ضروری به‌طور روشن و ساختارمند در اختیار مدل قرار گیرد. داده‌ها در این پرامیت به چند بخش تقسیم می‌شوند:

۱. متاداده‌ها (اطلاعات کلی پروژه).
۲. خلاصه‌ای از نتایج تست‌ها (شامل تعداد تست‌های موفق و ناموفق).
۳. مسیرهای بحرانی (آسیب‌پذیرترین بخش‌های کد که مستقیماً با شکست‌های تست مرتبط هستند).
۴. Hotspots (توابع یا نواحی کد که بیشتر در مسیرهای بحرانی تکرار می‌شوند).

```

1 {
2   "meta": {
3     "project_path": "C:\\Users\\Mhadese\\ViderProjects\\Vik-SE\\Service",
4     "log_path": "C:\\Users\\Mhadese\\ViderProjects\\Vik-SE\\Service\\Application.Tests\\TestResults\\TestResults.xml",
5     "language": "csharp",
6     "graph": {
7       "nodes": 344,
8       "edges": 926
9     }
10  },
11  "summary": {
12    "total_tests": 44,
13    "executed_tests": 23,
14    "passed_tests": 20,
15    "failed_tests": 3,
16    "skipped_tests": 0,
17    "untagged_tests": 23,
18    "failed_detail": [
19      {
20        "name": "Application.Tests.ServiceTests::Application.Tests.ServiceTests.EventServiceTests.cs::EventServiceTests.GetAsync_WhenExists_returnEvent",
21        "file": "Application.Tests.ServiceTests.EventServiceTests.cs",
22        "line": null,
23        "error": "Expectation has property AnyoneCanAddSelf that the other object does not have,\\nExpectation has property Attachments that the other object does not have,\\nExpectation has property Attendees that the other object does not have",
24        "doc": ""
25      }
26    ]
27  },
28  "critical_paths": [
29    {
30      "error": "Expectation has property AnyoneCanAddSelf that the other object does not have,\\nExpectation has property Attachments that the other object does not have,\\nExpectation has property Attendees that the other object does not have",
31      "upstream": [],
32      "downstream": [
33        {
34          "node": "Application.Tests.ServiceTests::Application.Tests.ServiceTests.EventServiceTests.cs::EventServiceTests.GetAsync_WhenExists_returnEvent",
35          "type": "test",
36          "file": "Application.Tests.ServiceTests.EventServiceTests.cs",
37          "line": 22,
38          "is_test": true,
39          "status": "failed",
40          "docstring": ""
41        },
42        {
43          "node": "Infrastructure.Integrations::Infrastructure.Integrations.GoogleEventProvider.cs::GoogleEventProvider.GetAsync",
44          "type": "function",
45          "file": "Infrastructure.Integrations.GoogleEventProvider.cs",
46          "line": 17,
47          "is_test": false,
48          "status": "not_executed",
49          "docstring": ""
50        },
51        {
52          "node": "Domain.Exceptions::Domain.Exceptions.EventNotFoundException.cs::EventNotFoundException::EventNotFoundException",
53          "type": "function",
54          "file": "Domain.Exceptions.EventNotFoundException.cs",
55          "line": 9,
56          "is_test": false
57        }
58      ]
59    }
60  ]
61 }

```

شکل ۴-۱۲: نمونه خروجی استخراج نهایی پرامیت تولید شده توسط سامانه بخش اول

```

output > {}
critical_paths: {
  "Application.Tests.ServiceTests::Application.Tests.ServiceTests.EventServiceTests.cs::EventServiceTests::GetAsync_whenIdExists_returnEvent": {
    "downstream": {
      {
        "type": "test",
        "file": "Application.Tests.ServiceTests.EventServiceTests.cs",
        "line": 27,
        "is_test": true,
        "status": "failed",
        "docstring": ""
      },
      {
        "node": "infrastructure.Repositories::infrastructure.Repositories.UserScheduleRepository.cs::UserScheduleRepository::GetAsync",
        "type": "function",
        "file": "infrastructure.Repositories.UserScheduleRepository.cs",
        "line": 49,
        "is_test": false,
        "status": "not_executed",
        "docstring": ""
      },
      {
        "node": "external::where/1",
        "type": "external",
        "file": null,
        "line": null,
        "is_test": false,
        "status": "not_executed",
        "docstring": "",
        "origin": {
          "language": "csharp",
          "namespace": "infrastructure.Repositories",
          "usings": [],
          "symbol": "where",
          "kind": "call",
          "file": "infrastructure.Repositories.UserScheduleRepository.cs"
        },
        "external_key": "external::where/1"
      }
    ]
  }
},
hotspots: {
  "Application.Tests.ServiceTests::Application.Tests.ServiceTests.EventServiceTests.cs::EventServiceTests::GetAsync_whenIdExists_returnEvent": {
    "functions": {
      "Application.Services::Application.Services.EventService.cs::EventService::GetAsync",
      "Domain.Exceptions::Domain.Exceptions.EventNotFoundException.cs::EventNotFoundException::EventNotFoundException",
      "infrastructure.Repositories::infrastructure.Repositories.UserScheduleRepository.cs::UserScheduleRepository::GetAsync"
    },
    "files": {
      "Application.Services.EventService.cs",
      "Domain.Exceptions.EventNotFoundException.cs",
      "infrastructure.Repositories.UserScheduleRepository.cs"
    }
  }
}

```

شکل ۴-۱۳: نمونه خروجی استخراج نهایی پرامپت تولید شده توسط سامانه بخش دوم

این تقسیم‌بندی به مدل کمک می‌کند که به‌طور مؤثرتر و دقیق‌تر اطلاعات را تحلیل کند و به‌جای ایجاد یک پاسخ پراکنده، یک تحلیل منسجم از مشکلات کد و تست‌ها ارائه دهد. علاوه بر این، برخی از پرامپت‌ها شامل few-shot examples می‌شوند تا مدل بتواند با دید بازتری به مشکلات نگاه کرده و از تجارب قبلی برای بهبود تحلیل‌ها استفاده کند. `prompt_generator.py` نیز مسئول ترکیب دقیق این داده‌هاست و برای فرستادن به مدل، دستورالعمل‌های سطح بالایی را به پرامپت اضافه می‌کند. این فرآیند به‌طور خودکار نمونه‌های آموزشی یا همان few-shots را به پرامپت اضافه می‌کند که به مدل کمک می‌کند از پاسخ‌های قبلی برای یادگیری استفاده کند و پاسخ‌هایی با دقت بیشتر تولید کند.

You are an expert software QA/SE assistant. Analyze failed tests across Python/Java/C# projects, using the provided test summary, critical call paths, hotspots, and function snippets. Do rigorous internal reasoning but DO NOT reveal chain-of-thought. Output only the final JSON that follows the required schema. Explain root causes in detail, suggest fixes, and provide evidence-based rationale. For each failed test, you MUST provide a concise 'failure_type'. Prefer the simple exception name without package/namespace (e.g., AssertionError, AttributeError, NullPointerException). If no clear exception, choose a short category like: Timeout, Network, Configuration, Mocking, DataMismatch, Resource, Flaky, Other. Additionally, for each failed test, provide concise 'insight_bullets' (2-4 short bullets) that describe the aggregated/organizational impact if similar failures persist (e.g., stability, lead time, risk to release, tech debt hotspots). Keep them high-signal, non-generic, and consistent with severity/failure_type/context. Your tasks:

- 1) Identify which tests failed and what the exact error messages are.
- 2) Map failures to project areas using the provided critical paths and hotspots.
- 3) Produce a precise multi-sentence explanation for root-cause analysis per failed test.
- 4) Propose concrete fixes (code-level and/or config/integration).
- 5) If signals suggest flaky/environmental issues, state it explicitly.
- 6) Set 'failure_type' to a SHORT normalized label (exception simple name or category). Examples: AssertionError, AttributeError, NullPointerException, Timeout, Network, Configuration, Mocking, DataMismatch, Resource, Flaky, Other.
- 7) Provide 'insight_bullets' (2-4 short bullets) per test about project-level impact and why it matters.

Do not include chain-of-thought or step-by-step reasoning in the output; only final JSON.

TEST SUMMARY:

- Total: 24 | Passed: 2 | Failed: 1 | Skipped: 0
- Failures:
 - src/test/java/BookTest.java::BookTest::testGetTitle: expected:<TODAY IS THE DAY[.> but was:<TODAY IS THE DAY[]>

org.junit.ComparisonFailure: expected:<TODAY IS THE DAY[.> but was:<TODAY IS THE DAY[]>
at BookTest.testGetTitle(BookTest.java:55)

CRITICAL PATHS:

Test: src/test/java/BookTest.java::BookTest::testGetTitle
Error: expected:<TODAY IS THE DAY[.> but was:<TODAY IS THE DAY[]>
org.junit.ComparisonFailure: expected:<TODAY IS THE DAY[.> but was:<TODAY IS THE DAY[]>
at BookTest.testGetTitle(BookTest.java:55)

Downstream:

- src/test/java/BookTest.java::BookTest::testGetTitle → src/main/java/Book.java::Book::getTitle → src/main/java/AggUtils.java::AggUtils::getTextUppercase
- src/test/java/BookTest.java::BookTest::testGetTitle → src/main/java/Book.java::Book::getTitle → src/main/java/AggUtils.java::AggUtils::getTextUppercase
- src/test/java/BookTest.java::BookTest::testGetTitle → src/main/java/Book.java::Book::getTitle
- src/test/java/BookTest.java::BookTest::testGetTitle → external::assertEquals

شکل ۴-۱۴: نمونه خروجی نهایی ساخت پرامپت برای ارسال به مدل زبانی بخش اول

HOTSPOTS (locus hints):

- src/test/java/BookTest.java::BookTest::testGetTitle

functions: src/main/java/AggUtils.java::AggUtils::getTextUppercase, src/main/java/Book.java::Book::getTitle
files: src/main/java/AggUtils.java, src/main/java/Book.java

FUNCTION SNIPPETS (trimmed):

src/main/java/Book.java::Book::getTitle (line 5):

```
"""
...
    public String getTitle(){          return AggUtils.getTextUppercase(title);
...
"""
```

src/main/java/AggUtils.java::AggUtils::getTextUppercase (line 3):

```
"""
...
    public static String getTextUppercase(String text){          return text.toUpperCase();
...
"""
```

FEW-SHOT EXAMPLES (style only, no chain-of-thought):

INPUT:
[EXAMPLE — STYLE & STRUCTURE ONLY, DO NOT COPY ASSUMPTIONS]
GRAPH SUMMARY: 312 nodes, 655 edges.

TEST SUMMARY:

- Total: 120
- Executed: 120 (✅ 114, ❌ 3, ⚠️ 3)
- Failures detail:
 - com.acme.calc.CalculatorTest::sum_null_should_throw: NullPointerException

CRITICAL PATHS for each failed test:

Test: com.acme.calc.CalculatorTest::sum_null_should_throw
Error: java.lang.NullPointerException
Upstream:

- com/acme/calc/Calculator.java::sum → com/acme/calc/Numbers.java::sanitize

شکل ۴-۱۵: نمونه خروجی نهایی ساخت پرامپت جهت ارسال به مدل زبانی بخش دوم

```

Upstream:
  • com/acme/calc/Calculator.java::sum → com/acme/calc/Numbers.java::sanitize
Downstream:
  • com/acme/calc/CalculatorTest::sum_null_should_throw

FUNCTION SNIPPETS:

com/acme/calc/Calculator.java::sum (line 52):
  public Integer sum(Integer a, Integer b) {
    |   return a + b; // no null checks
    |
  }

com/acme/calc/Numbers.java::sanitize (line 19):
  public Integer sanitize(Integer x) { return x; }
OUTPUT:
{
  "analysis": [
    {
      "test_name": "com.acme.calc.CalculatorTest::sum_null_should_throw",
      "error": "NullPointerException",
      "call_path": ["com/acme/calc/Calculator.java::sum", "com/acme/calc/Numbers.java::sanitize"],
      "rationale": [
        "1) sum does not perform a null-check and directly adds a + b",
        "2) sanitize does not handle null values properly",
        "3) summing with null results in a NullPointerException"
      ],
      "root_cause": "Lack of null checks in Calculator::sum",
      "suggested_fixes": [
        "Add null-checks in Calculator::sum and throw IllegalArgumentException or return default value",
        "Improve sanitize to handle null values properly"
      ]
    }
  ]
}

```

شکل ۴-۱۶: نمونه خروجی نهایی ساخت پرامپت جهت ارسال به مدل زبانی بخش سوم

```

RESPONSE FORMAT:
Return JSON only. Do not add commentary, markdown fences or chain-of-thought. Keep 'rationale' to ≤3 short bullets; keep 'insight_bullets' to 2-4 concise
Schema:
{
  "analysis": [
    {
      "test_name": "<string>",
      "error": "<string>",
      "call_path": [
        "<node_1>",
        "..."
      ],
      "locus": {
        "files": [
          "<relpath>",
          "..."
        ],
        "functions": [
          "<file::Class::func>",
          "..."
        ]
      },
      "failure_type": "<short normalized type, e.g. AssertionError | AttributeError | NullPointerException | Timeout | Network | Configuration | Mocking | ...>",
      "root_cause": "<precise multi-sentence explanation>",
      "severity": "<low|medium|high>",
      "suggested_fixes": [
        "<actionable fix 1>",
        "<actionable fix 2>"
      ],
      "rationale": [
        "<very short bullets, no CoT>",
        "..."
      ],
      "insight_bullets": [
        "<impact bullet 1>",
        "<impact bullet 2>",
        "..."
      ]
    }
  ]
}

```

شکل ۴-۱۷: نمونه خروجی نهایی ساخت پرامپت جهت ارسال به مدل زبانی بخش چهارم

ماژول `avalai_client.py`، که به عنوان واسط ارتباطی با مدل های LLM عمل می کند، وظیفه ارسال پرامپت به مدل و دریافت پاسخ ها را بر عهده دارد. این ماژول با استفاده از API های موجود، داده های پرامپت ساختاریافته را به مدل ارسال می کند و نتیجه را دریافت می کند. خروجی مدل می تواند شامل پیشنهادات برای اصلاح کد، تحلیل ریشه ای خطاها، و پیشنهادات برای بهبود کیفیت کد باشد. این مرحله از فرآیند می تواند به توسعه دهندگان در شناسایی نقاط ضعف کد کمک کرده و در نهایت موجب بهبود کیفیت نرم افزار شود.

```

class AvalAIClient:
    def __init__(
        self,
        model: str = "gpt-4o",
        *,
        api_key_env: str = "AVALAI_API_KEY",
        base_url_env: str = "AVALAI_BASE_URL",
        default_base_url: str = "https://api.avalai.ir/v1",
        temperature: float = 0.2,
        max_tokens: int = 2000,
        system_profile: Optional[str] = None,
    ) -> None:
        api_key = os.getenv(api_key_env)
        if not api_key:
            raise RuntimeError(f"Set your {api_key_env} environment variable")
        base_url = os.getenv(base_url_env, default_base_url).rstrip("/")
        self.client = OpenAI(api_key=api_key, base_url=base_url)
        self.model = model
        self.temperature = temperature
        self.max_tokens = max_tokens
        self.system_profile = (system_profile or _DEFAULT_SYSTEM).strip()

    def generate(self, prompt: str, *, retries: int = 3) -> str:
        logging.info(
            f"[AvalAI] base_url={self.client.base_url} model={self.model} "
            f"temp={self.temperature} max_tokens={self.max_tokens}"
        )
        delay = 1.5
        last_err: Optional[Exception] = None
        for _ in range(retries):
            try:
                resp = self.client.chat.completions.create(
                    model=self.model,
                    messages=[
                        {"role": "system", "content": self.system_profile},
                        {"role": "user", "content": prompt},
                    ],
                    temperature=self.temperature,
                    max_tokens=self.max_tokens,
                )
                return (resp.choices[0].message.content or "").strip()
            except (APITimeoutError, RateLimitError, APIError) as e:
                detail = getattr(e, "response", None)
                if detail is not None:
                    try:
                        payload = detail.json()
                    except Exception:
                        payload = {}
                    err = payload.get("error") or {}
                    err_code = err.get("code")
                    err_msg = err.get("message") or str(e)
                    logging.error(f"[AvalAI] API error (code={err_code}): {err_msg}")
                last_err = e
                time.sleep(delay)
                delay *= 2
        if last_err:
            raise last_err
        return ""

```

شکل ۴-۱۸: کد ماژول `avalai_client.py`

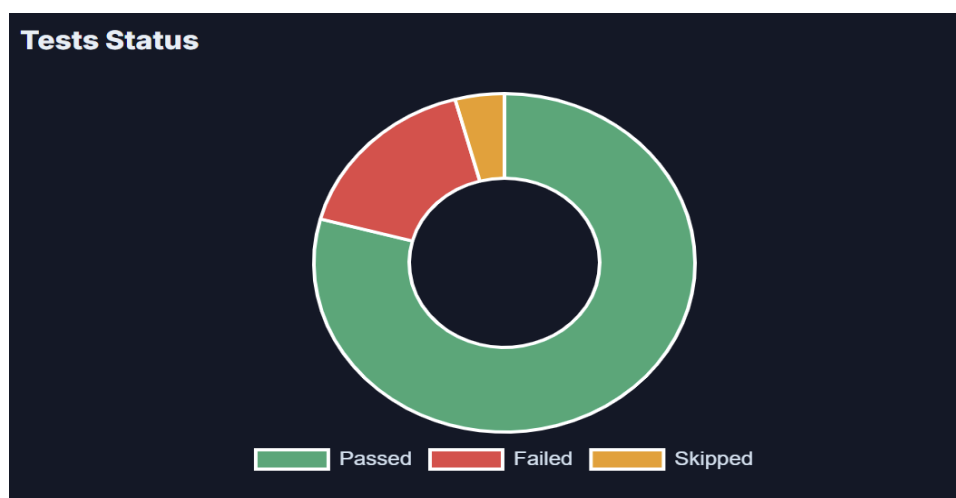
این بخش از سیستم، با استفاده از LLM، سطح جدیدی از تحلیل و بازخورد را ارائه می‌دهد که با روش‌های سنتی تست و تحلیل کد قابل مقایسه نیست. سامانه Failtrace از این قابلیت استفاده می‌کند تا یک گام جلوتر از روش‌های معمول تحلیل خطا باشد. با این حال، این فرآیند نیاز به تنظیمات دقیق در انتخاب داده‌های پرامپت و طراحی ورودی‌های مدل دارد، چرا که مدل‌های زبانی تنها زمانی مؤثر خواهند بود که داده‌های ورودی به‌طور دقیق و درست ساختاربندی شده باشند.

۴-۵-۶- تولید گزارش نهایی

پس از پردازش و تحلیل تمامی داده‌ها در مراحل مختلف، گام آخر در سیستم Failtrace تولید گزارش نهایی است که به‌طور جامع وضعیت تست‌ها و مسیرهای بحرانی را همراه با تحلیل‌های مدل‌های زبانی بزرگ در اختیار تیم توسعه قرار می‌دهد. این گزارش به‌صورت یک فایل HTML با اطلاعات جامع و نمودارهای گرافیکی تولید می‌شود و تمامی نتایج به‌صورت خوانا و کاربردی برای تحلیل دقیق‌تری از شکست‌های تست و کیفیت کد ارائه می‌شود.

ماژول `report_renderer.py` این فرآیند را با استفاده از داده‌های ساختاریافته‌ای که در مراحل قبلی تولید شده‌اند، انجام می‌دهد. گزارش نهایی در قالب HTML تولید می‌شود و شامل اطلاعات اصلی از قبیل تعداد تست‌های انجام‌شده، تعداد تست‌های موفق، شکست‌خورده، و ناتمام است.

برای نمایش وضعیت کلی پروژه، یک نمودار Doughnut Chart برای KPIها (مثل تعداد تست‌ها و وضعیت آنها) در گزارش گنجانده می‌شود. این نمودار با استفاده از کتابخانه‌ی `Chart.js` ساخته شده و به‌صورت خودکار با نتایج به‌روز می‌شود. در این نمودار، مرکز نمودار نشان‌دهنده وضعیت کلی تست‌ها (موفق، شکست‌خورده و ناتمام) است که مطابق شکل ۴-۱۸ نمایی کلی از کیفیت پروژه را به نمایش می‌گذارد.



شکل ۴-۱۹: نمودار آماری Doughnut Chart برای تحلیل نتایج یک پروژه نرم‌افزاری

علاوه بر آن، یک نمودار Bar Chart برای نمایش تعداد شکست‌ها و نوع آن‌ها در بخش‌های مختلف کد (مثل FailureTypes) به صورت نموداری گرافیکی قرار می‌گیرد. این نمودار کمک می‌کند تا به راحتی بتوانید محل و نوع خطاها را شناسایی کرده و از آن برای بهبود بخش‌های مختلف پروژه استفاده کنید (شکل ۴-۱۹).



شکل ۴-۲۰: نمودار آماری میله‌ای (Bar Chart) برای نتایج آزمون در یک پروژه نرم‌افزاری

علاوه بر نمودارها، جدول‌های متنی نیز در گزارش درج می‌شود که در آن‌ها جزئیات دقیق هر تست، مسیرهای بحرانی، توابع کلیدی و پیام‌های خطا آورده شده است. جدول Failed Items شامل ستون‌هایی از جمله «Test Name»، «Error Message»، «Failure Location» و «Suggested Fixes» است که به طور دقیق تمامی خطاها و مشکلات را شناسایی کرده و پیشنهادات اصلاحی را برای هر یک از آن‌ها در نظر می‌گیرد. این اطلاعات به توسعه‌دهندگان کمک می‌کند که سریع‌تر به مشکلات کلیدی کد پی ببرند و از لحاظ اولویت‌بندی بهینه‌تر عمل کنند (شکل ۴-۲۰).

Failed Items			
Root Cause	Location	Error	Suggested Fixes
The test fails due to mismatched properties between the expected and actual EventResponse objects, likely caused by incomplete or inconsistent mapping logic in GoogleEventProvider::GetAsync.	Priority 1 Graph Match Application.Services Infrastructure.Integrations Priority 2 Error Clues Application.Services Infrastructure.Integrations Priority 3 LLM Suggestion Application.Services Infrastructure.Integrations	Expectation has property AnyoneCanAddSelf that the other object does not have. Expected property result.Data.ColorId to be <null>, but found "".	Review and update the mapping logic in GoogleEventProvider::GetAsync to ensure all expected properties are populated correctly. Add validation checks in EventService::GetAsync to handle null or default values for critical fields.

شکل ۴-۲۱: جدول Failed items در آزمون یک پروژه نرم‌افزاری

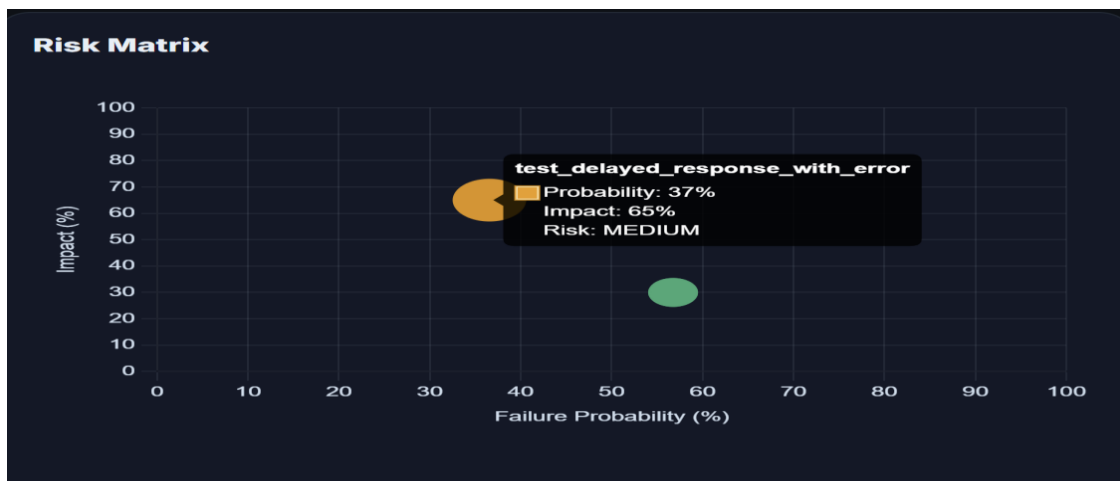
در این گزارش، اطلاعات تحلیل خطاها در قالب جدول Insight نمایش داده می‌شود. این جدول جزئیات تحلیل‌های ریشه‌ای هر تست را به‌طور دقیق بیان می‌کند و دلایل شکست تست‌ها و نکات کلیدی را نشان می‌دهد. برای هر خطا، جدول اطلاعاتی از جمله نام تستی که شکست خورده است و نقاط بینش که نشان‌دهنده تأثیرات آن خطا بر پروژه هستند، شامل ریسک‌های ممکن، تهدید برای کیفیت پروژه یا کاهش سرعت تحویل، ارائه می‌کند. این جدول به توسعه‌دهندگان این امکان را می‌دهد که سریعاً مشکلات بحرانی پروژه را شناسایی کرده و برای رفع آن‌ها اولویت‌بندی کنند (شکل ۴-۲۱).

Insights	
Concise, high-signal observations derived from the LLM analysis.	
Insight	Detail
test_will_fail	- Incorrect logic in core functionality risks cascading failures in dependent tests. - Fixing this issue ensures correctness and reduces debugging overhead in future tests.
test_delayed_response_with_error	- Misconfigured mocks can lead to unreliable test results and wasted debugging time. - Ensuring consistent mock setups improves test reliability and reduces lead time for releases.

شکل ۴-۲۲: جدول اطلاعات تحلیل خطاها

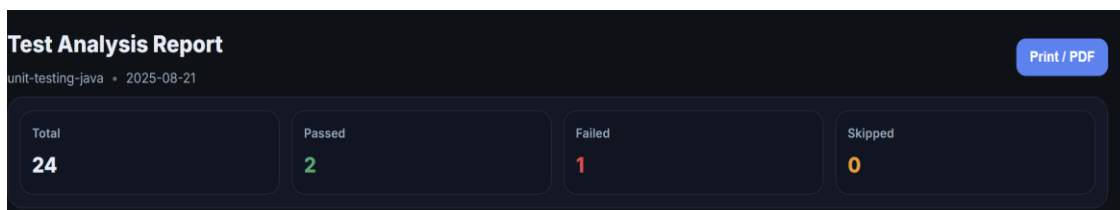
نمودار حبابی که در این گزارش به نمایش درآمده است، به‌صورت گرافیکی توزیع خطاها و توابع کلیدی را نشان می‌دهد. هر حباب در نمودار نمایانگر یک تابع یا بخش خاص از کد است که در آن خطاهایی رخ داده است. اندازه هر حباب نشان‌دهنده تعداد خطاها در آن بخش است و رنگ حباب نیز می‌تواند نمایانگر شدت آن‌ها باشد. این نمودار به تحلیلگران کمک می‌کند تا به‌راحتی بخش‌هایی از کد که بیشترین تعداد خطا را دارند شناسایی کنند و اولویت‌بندی بهتری برای رفع این مشکلات انجام دهند.

ویژگی‌های این نمودار شامل اندازه حباب‌ها است که نمایانگر تعداد خطاها در هر بخش از کد می‌باشد و رنگ‌های مختلف که می‌توانند برای نمایش نوع خطا (مثلاً قرمز برای خطاهای بحرانی، زرد برای خطاهای متوسط و سبز برای خطاهای کم ریسک) استفاده شوند. همچنین، نمودار حبابی دارای toolbar است که به کاربر این امکان را می‌دهد تا ویژگی‌های مختلف نمودار را بررسی کند (شکل ۴-۲۲).



شکل ۴-۲۳: نمودار حبابی مصورسازی ماتریس ریسک

برای ذخیره‌سازی و دسترسی سریع، فایل گزارش تولیدشده به‌طور خودکار در مسیر مشخص‌شده ذخیره می‌شود و امکان باز کردن آن در مرورگر به‌صورت مستقیم نیز فراهم است. این گزارش می‌تواند به‌راحتی به‌صورت PDF تبدیل شده و یا برای سایر اعضای تیم ارسال شود. در نهایت، این گزارش به‌عنوان یک ابزار بسیار مفید در فرآیندهای CI/CD و یکپارچه‌سازی مداوم در تیم‌های توسعه استفاده می‌شود تا تحلیل‌های ریشه‌ای خطاهای تست به‌طور سریع و مؤثری انجام شود (شکل ۴-۲۳).



شکل ۴-۲۴: نمایش وضعیت آماری کلی آزمون در یک پروژه نرم‌افزاری

۴-۵-۷- نحوه نصب و اجرای نرم‌افزار

یکی از مراحل کلیدی در به‌کارگیری هر سامانه نرم‌افزاری، آماده‌سازی صحیح محیط اجرا و نصب درست وابستگی‌هاست. ابزار Failtrace نیز به‌عنوان ابزاری برای تحلیل و پایش خطاهای تست نرم‌افزار، نیازمند طی شدن مراحل مشخص برای نصب و راه‌اندازی است. این ابزار در مخزن رسمی PyPI منتشر شده و به‌راحتی با استفاده از ابزارهایی مانند pip یا Poetry قابل نصب است. در این بخش، روند آماده‌سازی محیط، نصب و اجرای نرم‌افزار به‌صورت گام‌به‌گام توضیح داده می‌شود تا کاربران بتوانند بدون ابهام و با اطمینان از سامانه استفاده نمایند.

۴-۵-۷-۱- آماده‌سازی محیط اجرا

پیش از نصب، لازم است یک پوشه جدید برای پروژه ایجاد و وارد آن شد. سپس برای جلوگیری از تداخل کتابخانه‌ها، یک محیط مجازی^۱ ساخته می‌شود. محیط مجازی تضمین می‌کند که وابستگی‌های سامانه تنها در همین پروژه نصب و مدیریت شوند.

۱. ساخت پوشه جدید و ورود به آن:

```
mkdir project && cd project
```

۲. برای ایجاد محیط مجازی:

```
python -m venv venv
```

۳. فعال سازی محیط مجازی در ویندوز:

```
venv\Scripts\activate
```

۴. فعال سازی محیط مجازی در لینوکس و مک:

```
source venv/bin/activate
```

با فعال‌سازی محیط مجازی، اطمینان حاصل می‌شود که تمامی بسته‌های نصب‌شده تنها در این پروژه مورد استفاده قرار گرفته و از تداخل با سایر پروژه‌ها جلوگیری خواهد شد.

۴-۵-۷-۲- نصب ابزار Failtrace

بعد از آماده‌سازی محیط مجازی، گام بعدی نصب این ابزار است. این ابزار در مخزن رسمی PyPI منتشر شده و به راحتی از طریق pip قابل نصب است.

```
pip install failtrace
```

اجرای این دستور آخرین نسخه Failtrace را دریافت و نصب می‌کند. پس از اتمام نصب، ابزار آماده استفاده خواهد بود و می‌توان از قابلیت‌های آن برای تحلیل تست‌ها و ردیابی خطاها بهره گرفت.

^۱ Virtual Environment

۴-۵-۷ اجرای نرم افزار

نصب موفقیت آمیز سامانه امکان اجرای آن را از طریق رابط خط فرمان (CLI) فراهم می کند. Failtrace در دو حالت اصلی قابل اجرا است:

۱. حالت کامل

```
python -m failtrace full -p <PROJECT_PATH> -l <LOG_PATH> --lang
<LANGUAGE> --model <MODEL_NAME>
```

۲. حالت سریع

```
python -m failtrace quick -p <PROJECT_PATH> -l <LOG_PATH> --lang
<LANGUAGE> --model <MODEL_NAME>
```

خروجی اجرای دستور در محیط CLI به صورت شکل ۴-۲۵ نشان داده شده است. این تصویر نمایی کلی از اجرای Failtrace به همراه روند اجرا آن در رابط خط فرمان را نشان می دهد.

```
PS C:\Users\Mohadese\Desktop\Bachelor-Project\failtrace> python -m failtrace full --open-report -p C:\Users\Mohadese\Desktop\test-python-pytest\pytest-samples -l C:\Users\Mohadese\Desktop\test-python-pytest\pytest-samples\test-results\TEST-pytest-samples.xml
✓ Detecting language...
Completed in 0.41s (12% done)
✓ Building dependency graph...
Completed in 0.48s (25% done)
✓ Visualizing graph...
Completed in 0.01s (37% done)
✓ Loading test logs...
Completed in 0.01s (50% done)
✓ Tagging graph with logs...
Completed in 0.00s (62% done)
✓ Summarizing test results...
Completed in 0.00s (75% done)
✓ Building LLM structured prompt...
Completed in 0.19s (87% done)
✓ Extracting critical function code...
Completed in 0.02s (100% done)
✓ Sending prompt to AvalAI for analysis...
[AvalAI] base_url=https://api.avalai.ir/v1/ model=gpt-4o temp=0.2 max_tokens=2000
HTTP Request: POST https://api.avalai.ir/v1/chat/completions "HTTP/1.1 200 OK"
✓ Report generated: C:\Users\Mohadese\Desktop\Bachelor-Project\failtrace\failtrace\report\final_report.html
```

شکل ۴-۲۵: خروجی اجرای سامانه Failtrace در محیط خط فرمان

۴-۵-۷-۴ توضیح پارامترها

برای شفافیت بیشتر، پارامترهای موجود در دستورات فوق در جدول زیر معرفی شده‌اند:

جدول ۴-۲: پارامترها و گزینه‌های خط فرمان (CLI) سامانه Failtrace

آپشن	پارامتر	توضیح
--project	<PROJECT_PATH>	مسیر پروژه کد منبع (الزامی)
--log	<LOG_PATH>	مسیر فایل‌های گزارش تست‌ها (الزامی)
--lang	<LANGUAGE>	زبان برنامه‌نویسی پروژه (اختیاری)
--model	<MODEL_NAME>	نام مدل زبانی استفاده‌شده برای پردازش (اختیاری)

۴-۶- جمع‌بندی

در این فصل، مراحل طراحی و پیاده‌سازی سامانه‌ی تحلیل هوشمند نتایج تست نرم‌افزار تشریح شد. ابتدا نیازمندی‌های اصلی سیستم استخراج گردید و معماری کلی آن شامل ماژول‌های پردازش گزارش تست، ساخت گراف وابستگی، استخراج مسیرهای بحرانی و تولید پرامپت توضیح داده شد. سپس روند کلی اجزای سامانه و ارتباط آن‌ها با یکدیگر مورد بررسی قرار گرفت و پیاده‌سازی عملی اجزای کلیدی شامل ماژول تحلیل لاگ‌ها، تولید گراف و ساختاردهی خروجی‌ها به همراه نمونه کدها و نتایج اجرا ارائه شد. مجموعه این اقدامات نشان داد که سامانه پیشنهادی توانایی پردازش داده‌های پیچیده تست و تولید خروجی‌های قابل‌انکا برای پشتیبانی تصمیم‌گیری در فرایند تضمین کیفیت نرم‌افزار را دارد. به طور کلی، این پیاده‌سازی پایه‌ای عملی برای توسعه آتی سامانه فراهم ساخته و نشان می‌دهد که استفاده از مدل‌های زبانی بزرگ در کنار ابزارهای تحلیلی می‌تواند به بهبود سرعت، دقت و کارایی تحلیل نتایج تست کمک کند.

فصل پنجم

طرح تجاری

۵-۱- مقدمه

تجاری سازی سامانه‌ی تحلیل هوشمند نتایج تست نرم افزار، پاسخی به یکی از چالش‌های جدی صنعت توسعه‌ی نرم افزار است؛ چالشی که ریشه در افزایش پیچیدگی پروژه‌ها، رشد نیاز به تست‌های خودکار و دشواری تحلیل حجم بالای گزارش‌ها دارد. ابزار پیشنهادی این پژوهش، با تمرکز بر تست‌های واحد و یکپارچه سازی و بهره گیری از مدل‌های زبانی بزرگ (LLM)، نه تنها به توسعه دهندگان و تسترها امکان می‌دهد علل شکست تست‌ها را سریع تر و دقیق تر شناسایی کنند، بلکه با ساده سازی فرآیند تحلیل و کاهش هزینه‌های مرتبط با تضمین کیفیت، ظرفیتی بالقوه برای ورود به بازار نرم افزار و پذیرش گسترده در صنعت ایجاد می‌کند. هدف این فصل، تبیین ابعاد تجاری این سامانه و بررسی مسیرهای عملی برای عرضه‌ی موفق آن در بازار است.

۵-۲- جامعه هدف

جامعه هدف این پروژه طیف گسترده‌ای از فعالان حوزه نرم افزار را در بر می‌گیرد. در سطح نخست، تیم‌های توسعه و تضمین کیفیت (QA) در شرکت‌های متوسط و بزرگ قرار دارند که با حجم بالای تست‌های واحد و یکپارچه سازی مواجه‌اند و نیازمند ابزاری برای تحلیل سریع و دقیق نتایج تست هستند. استارت‌آپ‌ها نیز به دلیل محدودیت منابع مالی و انسانی، می‌توانند از این سامانه برای ارتقای فرآیند تضمین کیفیت بدون نیاز به تیم‌های بزرگ استفاده کنند. در صنایع حساس مانند بانکداری، سلامت دیجیتال و تجارت الکترونیک، که خطاهای نرم افزاری می‌تواند تبعات مالی و اعتباری جدی داشته باشد، ابزار پیشنهادی نقش حیاتی در افزایش اعتمادپذیری سیستم‌ها ایفا خواهد کرد. افزون بر این، مراکز دانشگاهی و پژوهشی نیز می‌توانند از سامانه به عنوان بستری آموزشی و تحقیقاتی در حوزه‌ی تست و تحلیل نرم افزار بهره‌مند شوند. بدین ترتیب، این پروژه نه تنها برای توسعه دهندگان و تسترها، بلکه برای طیف وسیعی از سازمان‌ها و نهادهای صنعتی و آموزشی ارزش عملی و کاربردی خواهد داشت.

۵-۳- سودرسانی

سودآوری پروژه‌ی حاضر که در حال حاضر به صورت متن‌باز و رایگان از طریق PyPI در اختیار توسعه‌دهندگان و تسترها قرار گرفته، نیازمند رویکردی تدریجی و آینده‌نگر است. همان‌گونه که تجربه‌ی ابزارهای مشابه مانند ReportPortal و Allure نشان می‌دهد، ابزارهایی از این جنس ابتدا با عرضه‌ی رایگان و متن‌باز جایگاه خود را در میان متخصصان پیدا کرده و سپس با ارائه‌ی خدمات حرفه‌ای و مدل‌های متنوع درآمدی مسیر تجاری‌سازی را طی کرده‌اند. پروژه‌ی حاضر نیز با پیروی از همین الگو، نخست بر جذب جامعه‌ی کاربری فعال و تثبیت جایگاه به‌عنوان ابزاری مرجع در تحلیل تست متمرکز خواهد شد. پس از ایجاد این جایگاه، مسیر حرکت به سمت درآمدزایی پایدار آغاز می‌گردد. در این راستا سه مسیر اصلی برای ایجاد ارزش اقتصادی و سودرسانی در نظر گرفته شده است:

۱. اشتراک ماهانه یا سالانه (SaaS/Cloud): در این مدل، نسخه‌ی ابری ابزار در اختیار تیم‌ها قرار می‌گیرد که شامل قابلیت‌های پیشرفته‌ای مانند داشبورد مدیریتی، یکپارچگی کامل با CI/CD، و ذخیره‌سازی تاریخچه بلندمدت تست‌ها خواهد بود. این نسخه به‌ویژه برای شرکت‌هایی که به دنبال سهولت استفاده و کاهش هزینه‌های نگهداری زیرساخت هستند جذاب است و بر اساس تعداد کاربران یا حجم داده قیمت‌گذاری می‌شود.
۲. تبلیغات هدفمند در نسخه رایگان: با گسترش جامعه کاربران نسخه‌ی رایگان، امکان نمایش تبلیغات تخصصی مرتبط با ابزارهای توسعه و تست فراهم خواهد شد. این روش، مشابه رویکرد بسیاری از سرویس‌های رایگان پایه^۱، به‌عنوان یک منبع درآمد مکمل عمل کرده و می‌تواند در کوتاه‌مدت بخشی از هزینه‌های توسعه و پشتیبانی پروژه را تأمین نماید.
۳. خدمات آموزشی و پشتیبانی تخصصی: یکی دیگر از مسیرهای کلیدی برای سودرسانی، ارائه دوره‌های آموزشی، مستندات تعاملی و وبینارهای تخصصی برای تیم‌های توسعه و تضمین کیفیت است. علاوه بر این، پشتیبانی سازمانی و مشاوره اختصاصی نیز به‌عنوان یک خدمت تجاری عرضه خواهد شد. این بخش به‌طور ویژه برای سازمان‌هایی ارزشمند است که قصد دارند از ابزار در مقیاس وسیع یا در محیط‌های حساس استفاده کنند. با چنین ساختاری، پروژه می‌تواند هم‌زمان جایگاه خود را در جامعه متن‌باز حفظ کرده و در عین حال مسیر درآمدزایی قابل اتکا و بلندمدت را طی کند. این ترکیب باعث می‌شود ابزار ضمن حفظ جذابیت برای جامعه توسعه‌دهندگان، از نظر اقتصادی نیز پایدار و توسعه‌پذیر باقی بماند.

^۱ Freemium

۵-۴- استراتژی بازاریابی و تبلیغات

موفقیت یک ابزار نرم‌افزاری متن‌باز، به‌ویژه در حوزه تست و تضمین کیفیت نرم‌افزار، صرفاً به توانمندی‌های فنی آن وابسته نیست؛ بلکه نیازمند بازاریابی هدفمند و تبلیغات مؤثر نیز می‌باشد. ابزار حاضر که در حال حاضر به‌صورت رایگان بر بستر PyPI در دسترس توسعه‌دهندگان و تسترها قرار گرفته، برای دستیابی به جایگاه تثبیت‌شده و تجاری‌سازی در مراحل بعدی، باید از همان ابتدا در معرض دید جامعه‌ی حرفه‌ای نرم‌افزار قرار گیرد. بر این اساس، دو مسیر اصلی برای بازاریابی و تبلیغات در نظر گرفته شده است:

۵-۴-۱- معرفی و ترویج از طریق محتوای آموزشی و شبکه‌های تخصصی

یکی از مهم‌ترین راهکارها برای معرفی ابزار در ایران، تولید محتوای آموزشی و حضور فعال در شبکه‌های حرفه‌ای است. انتشار آموزش‌های ویدئویی در پلتفرم‌هایی مانند آپارات و یوتیوب، نوشتن مقالات فنی در وبسایت‌هایی همچون ویرگول و مدیوم، و ارائه وبینارهای تخصصی در حوزه DevOps و QA Testing می‌تواند جامعه‌ای گسترده از توسعه‌دهندگان و تسترها را به ابزار جذب کند. در کنار این موارد، معرفی پروژه و ارائه راهنمای استفاده در بسترهایی مانند GitHub و StackOverflow و نیز تعامل فعال در گروه‌ها و کانال‌های تخصصی تلگرام و لینکدین باعث می‌شود ابزار در میان جامعه‌ی حرفه‌ای دیده شده و اعتماد کاربران به آن افزایش یابد. ترکیب این اقدامات، علاوه بر تبلیغ غیرمستقیم، نوعی برندینگ علمی-کاربردی برای ابزار ایجاد می‌کند که در فرآیند تجاری‌سازی آینده بسیار ارزشمند خواهد بود.

۵-۴-۲- همکاری با شتاب‌دهنده‌ها و شرکت‌های نرم‌افزاری

گام مهم دیگر، همکاری مستقیم با شرکت‌ها و شتاب‌دهنده‌های داخلی است. استارت‌آپ‌ها و تیم‌های نوپای نرم‌افزاری معمولاً منابع محدودی برای پیاده‌سازی تست و تضمین کیفیت دارند، از این‌رو ارائه نسخه رایگان یا آزمایشی ابزار به این گروه‌ها، ضمن رفع نیازهای آن‌ها، به‌عنوان تبلیغ عملی و مؤثر عمل می‌کند. همچنین، همکاری با شرکت‌های بزرگ نرم‌افزاری فعال در حوزه‌هایی چون بانکداری، تجارت الکترونیک یا مخابرات، این امکان را فراهم می‌سازد که ابزار در مقیاس سازمانی مورد استفاده قرار گیرد و بازخوردهای واقعی از محیط‌های پیچیده صنعتی دریافت شود. چنین همکاری‌هایی مسیر ورود ابزار به بازارهای سازمانی را تسهیل کرده و بستر لازم برای توسعه نسخه‌های تجاری و سازمانی^۱ را در آینده ایجاد می‌کند.

^۱ Enterprise

۵-۵- جمع بندی

تجاری سازی ابزار تحلیل هوشمند نتایج تست نرم افزار نیازمند برنامه ریزی دقیق و اجرای مرحله ای است. این پروژه با ارائه نسخه رایگان و متن باز در گام نخست، جایگاه خود را در میان توسعه دهندگان و تسترها تثبیت می کند و سپس با بهره گیری از مدل های درآمدی شامل اشتراک ابری، تبلیغات هدفمند و خدمات آموزشی، مسیر سودآوری پایدار را طی خواهد نمود. همزمان، با استفاده از استراتژی های بازاریابی مبتنی بر شبکه های حرفه ای، تولید محتوای آموزشی و همکاری با شتاب دهنده ها، زمینه معرفی گسترده ابزار و جذب جامعه کاربری فعال فراهم می شود. در مجموع، این ابزار با ترکیب نوآوری فنی و رویکرد تجاری سازی تدریجی می تواند به محصولی پایدار، کارآمد و اثرگذار در صنعت نرم افزار تبدیل گردد.

فصل ششم

نتیجه‌گیری و پیشنهادها

۶-۱- خلاصه

در دهه‌های اخیر با افزایش پیچیدگی نرم‌افزارها، رشد سیستم‌های چندلایه و گسترش روش‌های توسعه چابک، نقش تست و تضمین کیفیت بیش از پیش حیاتی شده است. حجم بالای تست‌های واحد و یکپارچه‌سازی و تنوع در سناریوهای تست، چالشی بزرگ در مسیر توسعه نرم‌افزار محسوب می‌شود. یکی از معضلات اصلی تیم‌های توسعه و تضمین کیفیت، تحلیل حجم انبوه گزارش‌های تست و شناسایی سریع دلایل شکست آن‌هاست؛ مسئله‌ای که اغلب موجب اتلاف زمان و افزایش هزینه‌های سازمانی می‌شود. پژوهش حاضر با هدف پاسخ به همین چالش طراحی گردید و بر آن بود تا با بهره‌گیری از ظرفیت مدل‌های زبانی بزرگ، ابزاری هوشمند ارائه کند که نه تنها فرآیند تحلیل تست را ساده‌تر سازد، بلکه امکان شناسایی ریشه‌ای خطاها و ارائه‌ی پیشنهادهای اصلاحی متناسب با پروژه را نیز فراهم آورد.

نتایج به‌دست‌آمده از پیاده‌سازی و آزمایش این سامانه نشان داد که استفاده از مدل‌های زبانی بزرگ توانسته است به شکل قابل ملاحظه‌ای سرعت تحلیل تست‌ها را افزایش داده و دقت در شناسایی ریشه‌ی خطاها را ارتقا بخشد. این موضوع نه تنها موجب کاهش وابستگی به تحلیل دستی و خطاپذیر می‌شود، بلکه نقش مهمی در کاهش بار کاری تیم‌های QA و توسعه ایفا می‌کند. همچنین، این ابزار توانسته است به‌عنوان یک چارچوب منسجم، فرآیند تست را از سطح صرفاً گزارش‌دهی به سطح تحلیل هوشمند ارتقا دهد و از این طریق بهره‌وری کل چرخه توسعه را افزایش دهد. دستاوردهای پژوهش نشان می‌دهد که چنین رویکردی ظرفیت بالایی در کاهش هزینه‌های مرتبط با تضمین کیفیت و بهبود اطمینان نرم‌افزارهای حیاتی دارد.

از منظر کاربردی و تجاری نیز این پروژه توانسته جایگاهی ارزشمند ایجاد کند. انتشار نسخه‌ی متن‌باز در بستر PyPI، دسترسی آسان توسعه‌دهندگان و تسترها را فراهم کرده و جامعه‌ای پویا پیرامون ابزار شکل داده است. این امر علاوه بر ایجاد زمینه برای بازخوردگیری و بهبود مستمر، بستری مناسب برای حرکت به سمت نسخه‌های سازمانی و حرفه‌ای ایجاد می‌کند. صنایع حساس همچون بانکداری، تجارت الکترونیک و سلامت

دیجیتال که نیازمند تحلیل دقیق و سریع نتایج تست هستند، می‌توانند از این ابزار بهره‌مند شوند و این مسئله چشم‌اندازی روشن برای تجاری‌سازی و توسعه پایدار آن ترسیم می‌کند. در مجموع می‌توان گفت که پژوهش حاضر با ارائه‌ی یک چارچوب هوشمند و عملی، هم از منظر علمی و هم از منظر صنعتی و اقتصادی ارزشمند بوده و ظرفیت ارتقا به یک محصول بین‌المللی پایدار را داراست.

۲-۶- چالش‌ها و محدودیت‌ها

یکی از مهم‌ترین چالش‌های فنی این پروژه، ضرورت پشتیبانی گسترده از ابزارهای مختلف تست و زبان‌های برنامه‌نویسی متنوع است. از آنجا که تیم‌های توسعه در صنایع و سازمان‌های مختلف از ابزارها و فریم‌ورک‌های متفاوتی استفاده می‌کنند (مانند JUnit، PyTest، NUnit یا Cypress) سامانه باید به گونه‌ای طراحی شود که قابلیت پردازش و تحلیل خروجی‌های گوناگون را داشته باشد. ایجاد چنین جامعیتی مستلزم طراحی ماژول‌های انعطاف‌پذیر و قابل توسعه است، اما همین موضوع موجب افزایش پیچیدگی فنی و دشواری نگهداشت سیستم می‌شود.

از سوی دیگر، تکیه بر مدل‌های زبانی بزرگ به صورت سرویس مبتنی بر API Key هرچند دسترسی سریع و بدون نیاز به زیرساخت محاسباتی سنگین را ممکن می‌سازد، اما محدودیت‌هایی نیز دارد. وابستگی به سرویس‌های خارجی و هزینه‌های مرتبط با استفاده از آن‌ها می‌تواند مانعی برای تیم‌های کوچک یا سازمان‌های داخلی باشد. علاوه بر این، مسائل مربوط به امنیت داده‌ها و محرمانگی گزارش‌های تست از دیگر چالش‌های جدی هستند؛ زیرا در بسیاری از صنایع حساس مانند بانکداری یا سلامت دیجیتال، امکان ارسال داده‌ها به سرویس‌های ابری خارجی با محدودیت‌های قانونی و امنیتی همراه است.

در بعد بازار و پذیرش، چالش اصلی رقابت با ابزارهای تثبیت‌شده‌ای همچون TestRail، ReportPortal و Allure TestOps است که سال‌ها تجربه، جامعه کاربری وسیع و قابلیت‌های تجاری متنوع دارند. در ایران نیز مسئله فرهنگ‌سازی اهمیت بالایی دارد، چراکه بسیاری از شرکت‌ها هنوز با ابزارهای سنتی کار می‌کنند و به لزوم تحلیل هوشمند نتایج تست به طور کامل واقف نیستند. بنابراین، ورود ابزار پیشنهادی به بازار داخلی و جهانی نیازمند تلاش گسترده در زمینه‌ی اعتمادسازی، آموزش و ارائه مزیت‌های عملی است تا بتواند جایگاه رقابتی خود را تثبیت کند.

۳-۶- کارهای آینده

بررسی نتایج حاصل از این پژوهش نشان می‌دهد که مسیر توسعه‌ی ابزار پیشنهادی همچنان باز و گسترده است و ظرفیت‌های متعددی برای ارتقا و تکمیل آن وجود دارد. نخستین جهت‌گیری در آینده، افزایش جامعیت

سامانه در پوشش طیف وسیع تری از نیازهای تست نرم افزار است. در این مسیر، تمرکز بر توانایی ابزار در پردازش و تحلیل انواع خروجی های تست و پشتیبانی از زبان ها و محیط های توسعه گوناگون ضروری خواهد بود. دستیابی به چنین هدفی می تواند جایگاه سامانه را از یک ابزار محدود به یک چارچوب جامع تحلیل و تضمین کیفیت ارتقا دهد و نقش آن را در زنجیره توسعه نرم افزار پررنگ تر نماید.

از سوی دیگر، یکی از محورها در آینده کاهش وابستگی به سرویس های خارجی و ارتقای استقلال فنی ابزار است. بهره گیری از مدل های زبانی بزرگ به صورت بهینه سازی شده و اختصاصی، امکان استفاده سازمانی و درون محیطی را فراهم کرده و مشکلاتی نظیر هزینه های بالا یا محدودیت های امنیتی را برطرف خواهد ساخت. همچنین، توسعه قابلیت های شخصی سازی در سطوح مختلف، مانند تعریف الگوهای تحلیل متناسب با نیاز هر تیم یا تنظیمات مرتبط با حساسیت سامانه نسبت به انواع خطاها، می تواند کارایی و اثربخشی آن را به طور قابل توجهی افزایش دهد.

در نهایت، آینده این پروژه تنها به جنبه های فنی محدود نمی شود، بلکه شامل ابعاد تجاری، آموزشی و اجتماعی نیز خواهد بود. گسترش ابزار در قالب نسخه های سازمانی، ارائه خدمات آموزشی و پشتیبانی تخصصی، و توسعه جامعه ای پویا از کاربران و پژوهشگران، مسیرهایی هستند که می توانند موجب تثبیت جایگاه ابزار در بازار نرم افزار شوند. علاوه بر این، حضور فعال در عرصه های علمی و صنعتی، مشارکت در همایش ها و کنفرانس ها و همکاری با مراکز نوآوری، بستری برای معرفی گسترده تر و توسعه پایدار سامانه فراهم خواهد آورد. چنین مسیری نشان می دهد که ابزار پیشنهادی نه تنها ظرفیت علمی و پژوهشی بالایی دارد، بلکه می تواند به محصولی مؤثر و ماندگار در صنعت نرم افزار تبدیل گردد.

منابع

- [1] M. Lutz, *Learning Python*, 5th ed., vol. 78, no. 1. Sebastopol, CA, USA: O'Reilly Media, 2007. doi: 10.1016/0019-1035(89)90077-8.
- [2] M. McGrath, HTML, CSS and JavaScript in Easy Steps. In *Easy Steps Limited*, 2020. [Online]. Available: <https://books.google.co.id/booksidYaSmxAEACAAI>
- [3] E. A. Meyer, *CSS: The Definitive Guide: The Definitive Guide*, vol. 7. O'Reilly Media, Inc., 2006. [Online]. Available: <https://books.google.com/books?id=rdtCRLXAL78C&pgis=1>
- [4] W. Wang, C. Yang, Z. Wang, Y. Huang, Z. Chu, D. Song, et al., "TESTEVAL: Benchmarking large language models for test case generation," in *Findings of the Association for Computational Linguistics: NAACL 2025*, Apr. 2025, pp. 3547–3562.
- [5] TestRail, "TestRail User Guide, " Gurock Software, 2023. [Online]. Available: <https://www.gurock.com/testrail/docs>. [Accessed: Aug. 24, 2025].
- [6] ReportPortal, "Official Documentation, " EPAM Systems, 2023. [Online]. Available: <https://reportportal.io/docs>. [Accessed: Aug. 24, 2025].
- [7] C. Beust, "TestNG Documentation, " 2023. [Online]. Available: <https://testng.org/doc/>. [Accessed: Aug. 2, 2025].
- [8] Allure Framework, "Allure Documentation, " Yandex, 2023. [Online]. Available: <https://docs.qameta.io/allure/>. [Accessed: Aug. 24, 2025].
- [9] Qameta Software, "Allure TestOps Documentation, " 2023. [Online]. Available: <https://qameta.io/alluretestops/docs>. [Accessed: Aug. 24, 2025].

پیوست ۱: لیست برنامه‌ها

مشاهده معماری کلی سیستم

مشاهده تصاویر خروجی سامانه